Jira Enhancer: A Governance-Aware Human-AI Collaboration System for Epic-to-Story Decomposition in Software Engineering

MANISH KUMAR AGRAWAL, Oracle India Pvt. Ltd., India and Indian Institute of Technology Roorkee, India
SANDEEP KUMAR, Indian Institute of Technology Roorkee, India
ASHOK SHUKLA, Oracle Corps, United States
SUBHAM KUMAR, Oracle India Pvt. Ltd., India
ABHISHEK GUPTA, Oracle India Pvt. Ltd., India
NANDAGOPAL SRINIVASAN, Oracle India Pvt. Ltd., India

Human–AI collaboration in software engineering now comprises coding, requirements enhancement, planning, and controlled updates to collaborative engineering artifacts, but unmanaged automated changes introduce severe risks to project governance and data integrity. We introduce Jira Enhancer, a system that converts sparsely specified epics into reviewable implementation stories while leaving human reviewers in complete control of issue-tracker state updates. The method combines context-aware evidence gathering across distributed repositories, hierarchical decomposition, automated refinement based on machine confidence tracking, bounded prompt-policy selection, explicit human approval gates, and conflict-safe writeback transactions. We evaluate authenticated field evidence from an industrial deployment at Oracle India Pvt. Ltd., tracking a team of 40 professional engineers across 14 large-scale epics, alongside an artifact evaluation blind to the source evidence, a matched baseline control experiment, a controlled fault-injection suite comprising 72,000 observations.

In two deployment phases, we recorded 70 terminal story endpoints: 69 implementation stories were developed, while a duplicate-safe matching pass successfully identified and linked one pre-existing record. In the matched baseline study, five independent reviewers evaluated 42 anonymized requirement packages created using 14 common inputs by a deterministic template baseline (B1), a one-shot drafting control (B2), and Jira Enhancer (B3). An exploratory analysis of the corrected reviewer ratings uses a revised composite emphasizing implementation readiness, dependencies and risks, and granularity and actionability. B3 scored 4.32 versus 3.59 for B2 on the 1–5 scale, a paired difference of 0.74 points (95% epic-bootstrap CI [0.55, 0.93], Holm-adjusted exact $p = 0.00024$). B3 exceeded B1 by 1.76 points (95% CI [1.43, 2.10], $p = 0.00024$) and scored higher than both baselines on all 14 epics. B3 received Ready-or-Minor-revision classifications in 88.1% of primary evaluations, compared with 66.7% for B2, with 5 versus 14 Major-revision ratings. Requirement assessment time was 2.15 minutes lower for B3 (95% CI [−2.35, −1.99], $p = 0.00024$). In a separate three-packet diagnostic, two blinded model evaluators recorded 0 unsupported additions and 0 missing source requirements for B3, compared with 4 and 13 findings for B2; ambiguous acceptance-criterion findings were 4 versus 21. These are summed model-evaluator findings. In independent fault-injection trials, B3's source-bound transaction envelope blocked all injected unsafe mutations while permitting all eligible writes under the declared fault model. Together, the evaluations support higher weighted requirement-quality ratings, a more favorable revision profile, and controlled writeback, with a higher computational budget than one-shot drafting.

CCS Concepts: • **Software and its engineering** → **Requirements analysis**; • **Human-centered computing** → **Collaborative interaction**; • **Computing methodologies** → *Reinforcement learning*.

Authors' addresses: Manish Kumar Agrawal, Oracle India Pvt. Ltd., Bengaluru, India, Indian Institute of Technology Roorkee, Roorkee, India, m_agrawal@cs.iitr.ac.in; Sandeep Kumar, Indian Institute of Technology Roorkee, Roorkee, India, sandeep.garg@cs.iitr.ac.in; Ashok Shukla, Oracle Corps, California, United States, ashok.shukla@oracle.com; Subham Kumar, Oracle India Pvt. Ltd., Bengaluru, India, subham.su.kumar@oracle.com; Abhishek Gupta, Oracle India Pvt. Ltd., Bengaluru, India, abhishek.ae.gupta@oracle.com; Nandagopal Srinivasan, Oracle India Pvt. Ltd., Bengaluru, India, nandagopal.srinivasan@oracle.com.

2026. Manuscript submitted to ACM

Additional Key Words and Phrases: Human-AI collaboration, requirements engineering, agile planning, agentic software engineering, human-in-the-loop AI, governed refinement, governance, confidence scoring

ACM Reference Format:
Manish Kumar Agrawal, Sandeep Kumar, Ashok Shukla, Subham Kumar, Abhishek Gupta, and Nandagopal Srinivasan. 2026. Jira Enhancer: A Governance-Aware Human-AI Collaboration System for Epic-to-Story Decomposition in Software Engineering. *ACM Trans. Softw. Eng. Methodol.* 1, 1 (November 2026), 44 pages.

1 INTRODUCTION

The topic of this research paper addresses the problem of human-AI cooperation in software engineering: how AI agents can transform incomplete planning intent into reviewable and execution-ready Jira stories while maintaining accountability, governance, and control over shared engineering records. The problem under consideration is associated with well-known requirements-engineering issues, including ambiguity, incompleteness, and stakeholder coordination. **These core issues present a major challenge in software engineering because incomplete tickets force engineering leads to waste time manually hunting for missing data across separate repositories, slowing down sprint execution cycles.** Teams do not need another channel that produces fluent text which must then be reconciled manually. They need support that fits how planning decisions are reviewed, challenged, approved, and audited. **Maintaining strict human authority and governance over shared engineering records is vital because any incorrect automated writeback to a project tracker instantly corrupts a team's core development roadmap, causing broken dependencies and task inflation.**

This problem involves machine reasoning, requirements engineering, software delivery operations, and enterprise governance. It is particularly relevant to organizations that want AI assistance but do not accept opaque automation, uncontrolled writeback, or weak accountability.

Recent software-engineering roadmaps also describe agentic systems as collaborators across the full development process, rather than only as code generators, and they stress trust, human values, and verifiable evaluation. We therefore treat Jira Enhancer as an AI-supported collaborative planning workflow rather than as a generator. The AI provides contextual information, decomposition suggestions, confidence estimates, and draft revisions. Human reviewers evaluate the evidence, approve or decline proposed stories, and retain ultimate control over external issue-tracker mutation. Such a framing is consistent with human-AI interaction literature that recommends systems reveal their capabilities, uncertainty, and user control at the point of decision. It also matches conclusions from studies of automation trust that emphasize the importance of appropriate rather than blind reliance on AI.

1.1 Research Scope and Research Questions

AI-supported planning is considered not as one-shot automation but as a joint socio-technical workflow. The object of study is the collaboration boundary: which stages of epic decomposition can be delegated to an AI agent, which stages require human decision-making, how confidence and evidence should be represented, and how shared software-engineering records can be changed without reducing accountability.

Our research focus centers on Epics and the Jira platform because they serve as the foundational source of truth where project requirements are defined and prioritized in commercial engineering environments. By evaluating specific variables such as machine confidence, evidence provenance, and programmatic policy gates, we seek to determine if a structured AI assistant can lower developer review burden without introducing duplication errors or stale data writes. Therefore, we investigate four research questions:

- **RQ1:** How can a human-AI workflow split underspecified software epics into reviewable implementation stories without compromising human authority over acceptance and writeback?
- **RQ2:** How can machine confidence, evidence provenance, and policy gates be combined to help reviewers determine whether AI-generated planning records are reliable enough for action?
- **RQ3:** What failure modes emerge when agentic planning systems interact with enterprise issue trackers, including missing evidence, duplicate story creation, partial writeback failure, and stale approvals?
- **RQ4:** How does a governed human-AI workflow perform, relative to executable template/rule and generic-LLM baselines, on decomposition quality, review effort, and implementation readiness?

The questions consider effective collaboration, joint performance, persistent agents, and new failure modes in software-engineering workflows. The contribution extends beyond implementation of the tool. It presents an executable study of how AI agents and human reviewers coordinate around real software-engineering records under explicit governance constraints **within a live enterprise footprint at Oracle India Pvt. Ltd., tracking the workflow behavior of 40 professional software practitioners across 14 large-scale epics.**

The remainder of this paper is organized as follows: Section 2 outlines related work and system motivation. Section 3 describes our system architecture, formal problems, and transaction models. Section 4 presents our evaluation design and dataset boundaries. Section 5 discusses our field results, baseline comparisons, and fault tests. Section 6 discusses the results and workflow trade-offs, and Section 7 presents the engineering lessons. Section 8 sets out practical implications and validity limits, and Section 9 concludes. Appendix A summarizes the scoring-model sanity checks.

2 RELATED WORK AND SYSTEM MOTIVATION

In modern software development, applications depend on an issue-tracking tool, such as Jira, to support execution. Although teams can capture high-level epics, these epics may be underspecified because they lack acceptance criteria, dependencies, and uniform estimation granularity. Previous work on agile requirements [11, 19], requirements standards [20], and issue-reporting quality [9] shows that structure, completeness, and actionability greatly influence subsequent engineering processes [36]. **In typical industrial landscapes, existing project management extensions and requirements tools generally fall into one of three distinct classes:**

- static templates with no intelligence, **which establish standard formatting layouts but do not dynamically improve text completeness or semantic quality [11, 20];**
- recommendation tools with no collaboration semantics or governance (high risk), **such as unconstrained large language model copilots that generate fluent prose but lack internal confidence validation loops [12, 14]; or**
- **automated workflow automation engines that route issues deterministically based on simple rules but are completely** unable to correlate confidence, review decisions, and write permissions (low trust).

This shortcoming is observed in everyday practice when doing simple things such as re-writing tickets during sprint planning, inferring hidden dependencies from distributed documentation by engineering leads, and dealing with unclear acceptance criteria by quality assurance during testing. **This makes a cooperative human-AI approach highly valuable, as an intelligent assistant can flag semantic gaps while a human developer guides the refinement loop to maintain process accountability.** The shortcoming is not limited to low process efficiency but rather to the inability to shape the information between capturing intentions and implementation. Similar issues were identified

in the bug reporting and issue triage research with regards to vague terminology and poor routing signals as factors increasing coordination cost [3, 21, 27]. Thus, the technical problem is to build a reliable machine system that can: (1) infer missing stories from an epic, (2) fill out stories until they become ready for implementation, (3) estimate confidence and uncertainty, (4) approve and enforce policies, (5) write results deterministically and auditably.

As a system designed to work in the real world has to deal with partial evidence, limited access to external systems, and multiple users editing things simultaneously, it should be able to run reliably both with clean data and with the enterprise noise.

2.1 Background from Existing Systems

The proposed solution will help solve shortcomings of the related systems of three classes. First, static authoring templates in project-management software offer better formatting but do not significantly enhance requirements completeness and decomposition quality in general. Second, advanced large language model copilots generate fluent texts but frequently lack formal confidence controls, policy coupling, and conflict-safe mutation policies. At last, automated workflow engines may handle deterministic state transitions, however, rarely employ prompt-based approaches or construct stories from evidence. These systems alone cannot help humans and AI cooperate in order to share context, manage uncertainty, recover from failure, and retain responsibility for co-created software-engineering records.

LLM and generative AI literature in software engineering and requirements engineering show good results for requirements engineering, issue resolution, and development workflows [12, 14]. At the same time, there are challenges in reliability, controllability, and evaluations of the existing approaches [12, 14]. Research on code generation assistants describes conditions showing a similar pattern: developers inspect, revise, and place generated outputs within the task context rather than accepting them unchanged [7, 52]. In case of agile planning, decomposition quality and review effort determine execution predictability, but governance-aware human oversight is still required for trustworthy deployments [2, 36, 50].

Multi-armed bandits and reinforcement learning algorithms are popular methods for selecting policies in an online setting under uncertainty [5, 30, 51]. Application of these algorithms to enterprise planning is difficult due to delayed and potentially unreliable rewards and the need to adhere to governance constraints. In turn, the suggested approach solves this problem by coupling an adaptive prompt policy to an auditable workflow state and explicit approval gates.

Transaction safety [8] and deterministic workflow control [53] are important aspects of writeback safety in LLM-based code generation. They are considered in LLM-based retrieval and tool-usage pipelines. Combining these aspects with RL-guided, LLM-based decomposition is intended to increase planning intelligence while preserving mutation safety in shared issue trackers.

Thus, the proposed approach cannot be considered solely a code generation advancement but rather its unique combination of search, decomposition, confidence scoring, policy evaluation, approval semantics, and safe writeback.

2.2 Related Work: Thematic Positioning

Theme A: Evidence retrieval and context construction. Studies on retrieval-augmented generation and dense retrieval [29, 46] show that evidence quality and the retrieval process are essential elements for response quality and hallucinations reduction [4, 17, 25, 33]. Jira Enhancer incorporates this insight in the form of context packing: rather than delivering evidence blocks to the generator, Jira Enhancer ranks the evidence and maps missing signals.

Theme B: Quality of reasoning via bounded refinement. Studies on reasoning-prompting and self-reflection [48, 54] show that it has positive impact when models are capable of deliberating, evaluating their outputs, and refining

Table 1. Comparison of the main paradigms discussed in the background and related work: representative strengths, enterprise-planning limitations, and the position of Jira Enhancer (B3) relative to B0/B1/B2.

Paradigm	Representative prior work	Typical strength	Common constraint in enterprise planning	Position of B0/B1/B2/B3
Template/rule decomposition	RE standards, agile requirements, and user-story quality work [11, 20, 36]	High consistency and stable formatting	Little ability to adapt to ambiguous epics; poor semantic understanding of unfamiliar situations	B0 and B1 are closest relatives of this family; B3 retains its structure but adds adaptive semantic reasoning
Generic LLM drafting	Prompt-driven generation, code assistants, and reasoning methods [7, 10, 14, 54]	natural-language drafts and broad transfer between tasks	Fluency, but lack of structure; confidence is uncoupled from mutation authority	B2 is closest relative of this family; B3 adds critic loops, validators and governance coupling
Retrieval-augmented generation	Retrieval-augmented and dense-retrieval methods [4, 24, 29]	Strong evidence base for grounding and factual memory recall	Varying retrieval quality; context-packing is not well defined for workflow-mutation situations	B3 converts retrieval into operation through ranking of context packets and explicit tracking of missing signals
Agent/tool orchestration	Tool-use methods and agent benchmarks [34, 35, 47, 58]	Strong multi-step execution ability and coordination with external systems	Inconsistent run-time behavior and risk of harm [43] when there is no deterministic bound on mutation	B3 constrains execution using policy-gated, hash-checked and idempotent write-back
Trust [28], confidence controls, governance	Alignment, human-AI collaboration, automation-trust, and socio-technical SE studies [6, 41, 42, 50]	Facilitates control and accountability [2]	Typically investigated in settings away from issue-tracker mutation	B3 directly implements this layer in Jira using human approval and policy gates

them according to some explicit criteria [16, 37, 59]. In line with this concept, Jira Enhancer uses a rubric-based critic along with a limited amount of retries. In other words, improvements occur according to explicitly defined scoring dimensions and not infinite prompting.

Theme C: Confidence, alignment, and reliable deployment. Work on alignment shows that fluent output alone is not a necessary condition of reliable performance in an important workflow context, without confidence controls and human interaction [22, 31, 41]. Furthermore, work in human-AI interaction and automation stresses the need for appropriate levels of trust, mental models, automation, and human control [2, 6, 28, 42]. Jira Enhancer resolves this issue by tying confidence to the policy gates and requiring approval before performing any ticket mutation.

Theme D: Constrained tools and safe execution. Studies on LLM tool use [60] demonstrate that models become more reliable when their outputs go through a constrained interface and verifiable tool steps [16]. Jira Enhancer implements this approach with deterministic writeback, which includes hashing, idempotency, and limited field modifications.

3 PROPOSED METHODOLOGY

The complete human-AI collaboration pipeline is structured as an interactive, state-guided loop designed to safely elevate requirements actionability within real-world engineering environments. The technical

method is split into five functional layers that process an under-specified corporate epic, enrich its technical contents, and cross-examine it against organizational policy boundaries before executing any tracker updates. The closed loop is made up of five layers: (1) scope resolution maps issue, sprint, epic, or release scopes to issue sets; (2) semantic enrichment develops structured descriptions, acceptance criteria, risks, dependencies, and next steps; (3) prompt selection chooses bounded refinement strategies and records attempt scores; (4) policy and governance apply thresholds, flags, and human-approval rules; and (5) writeback and audit perform bounded mutations with conflict checks and immutable traces. **The representative strengths, enterprise constraints, and theoretical position of Jira Enhancer relative to these multi-paradigm baseline configurations are summarized systematically in Table 1.**

First, the system generates stories, refines them toward a displayed confidence target (for example, **the target interface metric** $C_{10}(s) \geq 9$**, which scale-transforms normalized algorithm confidence** $C(s) \in [0, 1]$ **into a readable** 0–10 **metric via** $C_{10}(s) = 10C(s)$), and creates a candidate story only after individual approval through controlled field mappings. **The system's internal creation gate threshold is configured strictly at** $\tau_c = 0.9$ **(displayed as** $C_{10} = 9$ **on the user interface scale). This conservative choice serves as a corporate quality constraint, ensuring a candidate story exhibits a near-complete array of source evidence parameters before it is permitted to consume an engineer's review attention.** The AI retrieves evidence, drafts and scores candidate stories, and performs bounded refinement. The reviewer evaluates the context and risk but retains final control. Generation, review, and writeback share a common state rather than being distinct processes, consistent with socio-technical guidelines for technology assessment in context [49, 50]. The candidate is a stateful planning artifact, not just a disposable solution. The log contains the evidence, the choice of prompt arm, the variation in confidence, the intervention of the reviewer, and the policy decisions. Creative generation can be retried, but any mutation is permanent and limited by the policy decision for conflicts. This means that normal structuring can proceed while maintaining auditability of human for significant mutations.

3.1 API and UI Console Workflow

3.1.1 Workflow API. The API supports two connected tasks. The first task enriches existing Jira items through a human review and revision loop. The second task decomposes an epic into candidate stories. Jira mutation remains separate from both tasks.

Run creation and inspection. `POST /api/v1/runs` starts an enrichment run for an issue, sprint, epic, or release. `GET /api/v1/runs/{runId}` returns the overall run status. `GET /api/v1/runs/{runId}/items/{issueKey}` returns the current draft. It also returns the evidence, warnings, readiness, confidence, policy state, and prompt-selection details. These read endpoints allow a reviewer to inspect the result before making a decision.

Human guidance and revision. The reviewer does not need to accept the first draft. `POST /api/v1/runs/{runId}/items/{issueKey}/approval` records a decision for one item. The allowed values are `approve`, `reject`, `regenerate`, `user_input`, and `prompt`. The `user_input` decision carries a human suggestion. The `prompt` decision carries a reviewer-written instruction for further modification. Both forms of guidance are sent back to the reasoning process, which produces a revised draft.

The run-level endpoint, `POST /api/v1/runs/{runId}/approval`, applies the same decision and guidance to every item in the run. The system may also ask a clarification question. In this case, `POST /api/v1/runs/{runId}/items/`

`{issueKey}/interaction-answer` records the human answer and continues the revision. Human suggestions and follow-up prompts are therefore part of the API workflow.

Controlled writeback. `POST /api/v1/runs/{runId}/items/{issueKey}/writeback` writes one approved item after the policy, validation, source-conflict, and retry checks. `POST /api/v1/runs/{runId}/writeback` applies this operation across the run. Each item still receives its own result. This preserves item-level reasons for a successful, blocked, conflicting, or repeated request.

Epic story decomposition. `POST /api/v1/epics/{epicKey}/stories/suggest` reads the epic, existing child stories, and available evidence. It returns editable story candidates with readiness, confidence, threshold status, warnings, and evidence references. It does not create Jira stories.

`POST /api/v1/epics/{epicKey}/stories/create` receives the reviewed story payloads. It checks confidence and approval. It skips matching existing summaries and attempts to create only the remaining stories. Its response lists created and skipped items, including duplicate and creation-failure reasons.

The API therefore supports a full cycle: inspect, suggest, provide human guidance, revise, approve, and write back. Human users and controlled clients use the same contracts and governance checks.

3.1.2 Console Workflow. The console exposes the same workflow in three connected views. The enrichment view starts a run. It shows each Jira draft with its evidence, warnings, readiness, confidence, and selected prompt. The review view allows the user to approve, reject, request regeneration, provide a human suggestion, or submit a Jira-specific prompt for further modification. The same guidance can be applied to one Jira or to all Jira items in the run.

The **Epic Story Breakdown** view provides the following features:

(1) epic-key and project-key inputs,
(2) the suggestion action, which reads the epic and proposes child stories,
(3) an editable structured payload for reviewing each story, with an `approved` flag per story,
(4) the approved-story creation action for writing back qualified rows.

The reviewer can edit the generated content, add missing context, or provide a new prompt before approval. Writeback remains a separate action. This prevents a revision request from being treated as permission to change Jira.

Human-led and agent-led operations are both provided via the API. The human reviewer assesses the narrative and the possible risks through the console, whereas controlled agents or pipelines do that using the same endpoints according to deterministic payload contracts. This allows governed integration with release-planning bots, PM dashboards, and governance data lakes.

The UI focuses on "decision visibility," giving the reviewer the sources of confidence, warnings, policy reason codes, approval provenance, reasons for skipping duplicates, and preserved retry selections for each story candidate. This discourages reviewers from treating confidence as an unsupported number and enables quicker and better accept, reject, or retry decisions. The design seeks appropriate trust rather than maximum automation, reflecting evidence that human–AI team performance depends on users' mental models of system behavior [2, 6, 28].

3.1.3 Evidence Availability and Recovery. The system classifies warning codes, such as external authentication requirements and MCP unavailability, and propagates them to run- and story-level review records. This supports graceful degradation:

$$\texttt{service_mode} \in \{\texttt{full-context}, \texttt{reduced-context}\}. \tag{1}$$

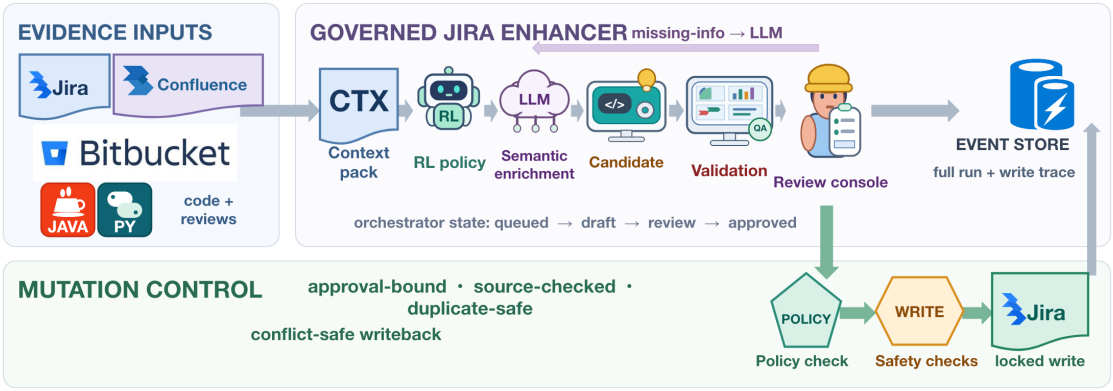


Fig. 1. Human-AI collaboration architecture. Jira, Confluence, and Bitbucket provide evidence in the form of issues, requirements, design, code, and reviews. The orchestrator controls evidence-based story creation, bounded prompt arm selection, governance checks, audit storage, and controlled writeback of Jira issues. The labels and arrows define the sequence of runtime contracts that starts from the context extraction and human actions and proceeds with scoring, policy check, approved candidate generation, and mutation.

Full-context mode is used for cases with external evidence; *reduced-context* mode is applied when only issue-local evidence is available.

Reduced-context mode is designed to remain useful instead of failing softly. It still provides structured drafts and warning metadata, which allows reviews to continue in an informed way.

In reliability engineering, the "graceful capability collapse" pattern is applied instead of binary failure. For example, if a high-value evidence source such as Confluence is offline, the system retains decomposition capability by using the available issue records and marking the missing evidence categories. Available issue records prevent planning deadlock, while metadata preservation maintains the meaning of the confidence score.

Transient failures are retried using exponential backoff and jitter, while terminal errors are reported with machine-actionable error codes. The technique allows for automatic retry management at the orchestration layer without hiding permanent integration issues.

The user-guided workflow maintains collaboration context through the stages in case of failures. If the story creation fails or times out, then the selected stories will still be present in the console. User will be able to look at the partial generation results in Jira and continue working on them without having to regenerate the decision set. Such capability is important for human–AI cooperation since recovery should not be limited to backend functionality but should be visible for a human making retry, change, or cancel decisions.

3.2 System Architecture

Figure 1 presents the complete system architecture, showing how the LLM, RL-guided prompt-selection layer, and human-governance mechanisms interact to address the software-engineering and requirements-engineering challenges discussed above. The subsequent sections explain the main architectural components in greater detail.

3.2.1 Reading Figures 1–3. The set of Figures 1–3 provides three perspectives: architecture and trust boundaries, operational sequence, and internal decomposition of agents. The deliberate overlap helps to conduct separate reviews of completeness of the architecture, correctness of the execution, and responsibilities of agents. **Crucially, the interface**

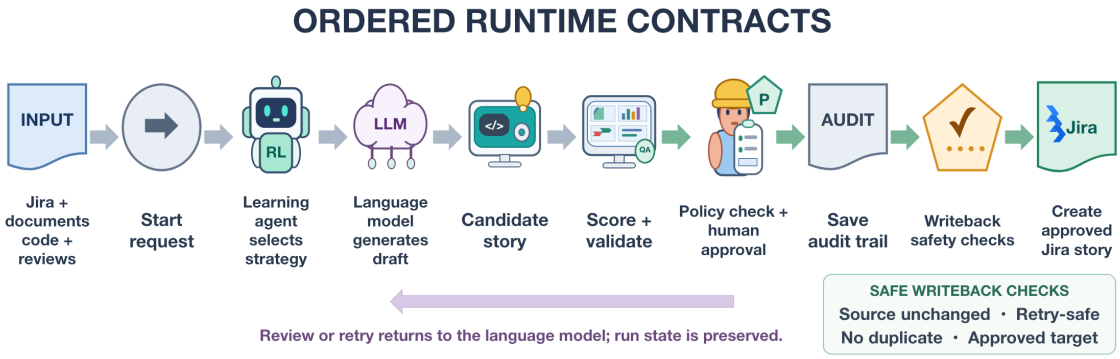
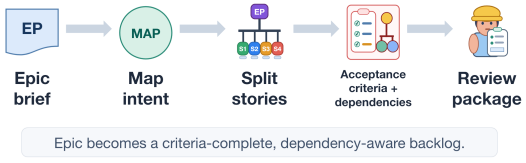


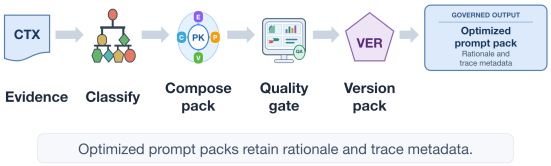
Fig. 2. Sequence of the runtime contracts according to Fig. 1. Jira, requirements, code, and review input is fed into the orchestrator. Safe prompt selection and scoring generate the candidates, policy check and tracing storage precede the creation of approved candidates; writeback is the last step of the process.



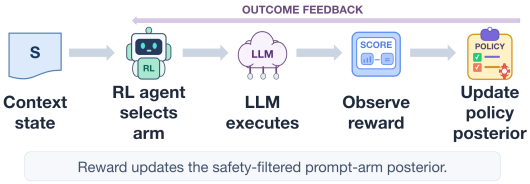
(a) Agentic module 1: story/issue enhancer.



(b) Agentic module 2: epic breaker into stories.



(c) Agentic module 3: prompt generator.



(d) Agentic module 4: RL controller.

Fig. 3. Modular agent architecture: story/issue enhancement, epic-to-story decomposition, prompt-pack creation, and RL-based policy control. The panels expose agent boundaries hidden in the top-level architecture.

junctions and directional arrows mapped across these graphics represent strict, programmatically enforced API contracts. System components interact across boundaries through rigid REST and schema serialization layers, ensuring that data handoffs are deterministically bound rather than relying on fluid text validation.

Figure 1: system architecture overview. Figure 1 places Jira epics, stories, and writeback targets; Confluence requirements and design evidence; and Bitbucket code and review signals outside the AI boundary. Inside Jira Enhancer, the review console, orchestrator, semantic enrichment engine, RL policy, governance layer, event store, and conflict-safe writer transform evidence into candidate stories. The architecture clearly separates AI reasoning from changes to Jira.

The enrichment and RL components can generate, evaluate, and improve a candidate story, but they cannot update Jira directly. A candidate can be written to Jira only after it passes validation and the confidence threshold, complies with the relevant policies, and receives human approval.

Figure 2: execution/control flow. Figure 2 displays the components in execution order. A run gathers Jira context and acquires requirement, design, code, and review signals. The orchestrator first makes a workflow entry and selects a permitted prompt strategy. Then, it generates and scores candidate stories, applies the policies, and logs all the steps. Only successful candidates may be written to Jira. Each arrow in the diagram depicts a clearly defined handoff from one component to another, which includes preparing inputs, logging the selection of prompts, candidate scoring, logging the policies applied, and validation of writing back to Jira. Before writing to Jira, the system ensures there are no conflicting edits, safe retries (idempotency), and duplicated stories. Otherwise, the item is left in review, refinement, conflict, or failure state without Jira being updated.

Figure 3: Internal agent modules. Figure 3 demonstrates four agentic capabilities within the AI engine. The first Agentic Module evaluates an issue and determines what acceptance criteria, testability, risks, and refinement areas are absent. The second Agentic Module creates candidate stories and dependencies for a high-level epic, adding acceptance criteria and data for human review. The third Agentic Module generates context-aware prompt packs using the issue text, reusable patterns, policy constraints, and information gaps that were identified. The fourth Agentic Module plays a role of an RL controller. It chooses an approved strategy to create prompts – policy arm, evaluates the outcome, rewards it according to its quality, and adjusts the posterior estimate that is a measure of future success of the corresponding strategy. It enables the choice between different strategies, such as conservative decomposition and balanced-context generation, while avoiding any unsafe or unrestricted prompts.

The captions in Figures 1 and 2 represent executable contracts. Context and Code Retrieve normalize the inputs, Human Actions captures the reviewer's intention, while selecting and generating safe-arms leads to scoring of candidates; Policy Check determines confidence, role, validation, and approval; Persist trace records state, results, and evidence links. Approved creation reaches controlled writeback only after the source hash, idempotency, duplicate, and target checks. The safety boundary is hence determined by this rule rather than fluency of text.

3.3 Traceable Data Records

Every run captures scope, requester, time stamps, and idempotency information along with item state, confidence, readiness, policy flags, structured drafts, references to external evidence, decisions and notes on approval, and before/after writeback hashes. Since each piece of information is a first class record, the planning decision behind it may be subsequently reconstructed for empirical evaluation [26, 44], auditing [39], or incident analysis. Immutable event streams maintain event ordering, whereas mutable snapshots facilitate low-latency querying. Informational warnings alert about non-critical deficiencies, advisory warnings bring problems to reviewer attention, and blocking warnings stop mutation until the issue gets fixed.

3.4 Formal Problem Formulation

Let E be an epic, S_{existing} the set of already-linked stories, $\mathcal{D}$ the available evidence corpus, and Π a versioned governance policy. **To model practical industrial lifecycles cleanly, the comprehensive evidence corpus is split into separate components: $\mathcal{D} = \mathcal{D}_{\text{ext}} \cup \mathcal{D}_{\text{jira}}$. Here, $\mathcal{D}_{\text{ext}}$ represents externally pulled context sources, which concretely include Confluence technical design document pages, corporate requirements specifications,**

and active Bitbucket code pull-request log branches. Conversely, $\mathcal{D}_{\text{jira}}$ isolates localized requirements constraints, containing issue-embedded discussion histories and text architecture summaries recorded directly inside the core Jira ticket text description field. This distinction is important because, when external links are unavailable but the epic description contains useful design rules, APIs, guardrails, or tests, the system can still produce evidence-grounded candidates while identifying their provenance as Jira-embedded context. The system constructs a finite candidate set S_{cand} and selects a subset $S_{\text{new}} \subseteq S_{\text{cand}}$ for possible creation. Here, requirements are modeled as evolving and negotiated objects rather than fixed text [40]. This agrees with established RE views that requirements are incomplete and shaped by stakeholders [38]. It also follows requirements-quality guidance on clarity, verifiability, feasibility, and traceability [20]. Each candidate s is represented by the tuple

$$m(s) = (C(s), R(s), U(s), \text{Amb}(s), \rho(s), A(s), h_s), \tag{2}$$

where $C(s)$ denotes normalized confidence, $R(s)$ denotes readiness, $U(s)$ is expected utility, $\text{Amb}(s)$ is ambiguity, $\rho(s)$ is risk, $A(s) \in \{0, 1\}$ is the binary human-approval indicator, and h_s is the source-state cryptographic hash generated when the decision is formally made. The notation $A_s = A(s)$ denotes the same approval indicator in the algorithms below.

Normalized story identity is modeled through an equivalence relation $\sim_n$:

$$s_i \sim_n s_j \iff \text{norm}(\text{summary}(s_i)) = \text{norm}(\text{summary}(s_j)), \tag{3}$$

Here, s_i and s_j are indexed stories, $\text{summary}(s)$ is the summary text of story s, and $\text{norm}(\cdot)$ converts text to lowercase and strips all non-alphanumeric separators from it. The duplicate-free candidate domain, denoted as $\mathcal{F}_{\text{dup}}$, is thus:

$$\mathcal{F}_{\text{dup}} = \{s \in S_{\text{cand}} \mid \nexists s' \in S_{\text{existing}} : s \sim_n s'\}. \tag{4}$$

Here, s' ranges over existing stories. The above guarantees that work item generation will not lead to duplicates caused by normalization, which would make different versions of task summaries identical.

The basic feasibility relation is:

$$S^* = S_{\text{existing}} \cup S_{\text{new}}, \tag{5}$$

$$\forall s \in S_{\text{new}} : \; C(s) \geq \tau_c, \tag{6}$$

$$\forall s \in S_{\text{new}} : \; A(s) = 1 \text{ (approved)}, \tag{7}$$

$$S_{\text{eligible}} = \{s \in S_{\text{new}} \cap \mathcal{F}_{\text{dup}} \mid C(s) \geq \tau_c \wedge A(s) = 1 \wedge V(s) = 1 \wedge \Pi(s) = \textsc{allow}\}, \tag{8}$$

$$\mathcal{W}_{\text{create}} = \text{Create}(S_{\text{eligible}}), \tag{9}$$

Here, S^* is the augmented plan, S_{eligible} is the subset satisfying all creation conditions, and $\mathcal{W}_{\text{create}}$ is the result of applying the controlled creation operation $\text{Create}(\cdot)$ to that subset. The normalized creation threshold is $\tau_c \in [0, 1]$ (default $\tau_c = 0.9$). The displayed confidence is $C_{10}(s) = 10C(s)$, so the default threshold corresponds to 9/10. The term $V(s) \in \{0, 1\}$ is the validation indicator for candidate story s. It holds true (i.e. 1) only if the candidate passes schema, contradiction, and required field test. The symbol Π denotes the versioned governance policy, and $\Pi(s)$ is its decision for candidate s. Condition $\Pi(s) = \textsc{allow}$ represents that the governing policy of the machine allows creation of the candidate.

This expression contains one of the important principles: high quality of generation does not necessarily guarantee the mutation. Creation is an action demanding both trust of the machine and of the human being, so that recommendation cannot become an action without permission.

Feasibility rules defined above specify which candidates are eligible for the creation phase. Yet, several of the candidates will satisfy all those rules and the machine still needs to determine which of them must enter the enhanced plan.

For each of the candidates $s \in S_{\text{cand}}$, we will define a binary decision variable $y_s \in \{0, 1\}$, where $y_s = 1$ if the candidate is selected and $y_s = 0$ if it is not selected. With non-negative weights λ_ρ and λ_{Amb} controlling the penalties for risk and ambiguity, the system uses the following objective:

$$\max_{\{y_s\}} \sum_{s \in S_{\text{cand}}} y_s \Big(U(s) - \lambda_\rho \rho(s) - \lambda_{\text{Amb}} \, \text{Amb}(s) \Big), \tag{10}$$

The objective is subject to the following constraints, where $\mathbb{1}[\cdot]$ equals one when its condition is met and zero otherwise:

$$y_s \le \mathbb{1}[C(s) \ge \tau_c], \tag{11}$$

$$y_s \le A(s), \tag{12}$$

$$y_s \le V(s), \tag{13}$$

$$y_s \le \mathbb{1}[\Pi(s) = \textsc{allow}], \tag{14}$$

$$y_s \le 1 - \mathbb{1}[\exists s' \in S_{\text{existing}} : \text{norm}(\text{summary}(s)) = \text{norm}(\text{summary}(s'))], \tag{15}$$

$$y_s \in \{0, 1\}. \tag{16}$$

Objective favours candidates with high expected utility $U(s)$. Selection score decreases if the candidate has high risk $\rho(s)$ or high ambiguity $\text{Amb}(s)$.

Constraints make it possible to select a candidate only when its confidence level reaches τ_c, it has human approval, it has passed validation, and active policy allows for its creation. The duplicate constraint makes it impossible to select a candidate whose normalized summary coincides with an existing story.

Thus, expected utility makes it possible to rank suitable candidates but cannot overcome confidence, approval, validation, policy, or duplicate conditions.

In scheduling applications, the objective can be extended with portfolio constraints:

$$\sum_{s \in S_{\text{cand}}} y_s \, \text{SP}(s) \le B_{\text{sprint}}, \tag{17}$$

$$\sum_{s \in S_{\text{cand}}} y_s \, \mathbb{1}[\text{team}(s) = g] \le \text{cap}_g, \tag{18}$$

$$\text{dep}(s_i, s_j) = 1 \Rightarrow \text{ord}(s_i) < \text{ord}(s_j), \tag{19}$$

where $\text{SP}(s)$ is the story-point estimate for candidate s and B_{sprint} is the sprint budget in story points. The function $\text{team}(s)$ gives the team assigned to story s, g identifies a team, and cap_g is its maximum number of selected stories. The dependency indicator $\text{dep}(s_i, s_j)$ equals one when story s_j depends on story s_i, and $\text{ord}(s)$ gives the release-order position of story s. This demonstrates that the proposed mechanism can support both decomposition quality and downstream planning feasibility.

Two invariants follow directly from the constraints:

$$y_s = 1 \Rightarrow C(s) \geq \tau_c, \tag{20}$$

$$y_s = 1 \Rightarrow A(s) = 1 \wedge V(s) = 1 \wedge \Pi(s) = \textsc{allow}. \tag{21}$$

Consequently, selected stories are both confidence-feasible and governance-feasible. These invariants support the multi-gate creation rule used later in the policy layer.

Proposition 1: eligibility soundness. For any candidate $s \in S_{\text{cand}}$, if $y_s = 1$ in the constrained optimization problem above, then s is confidence-feasible, approval-feasible, validation-feasible, policy-feasible, and duplicate-free:

$$y_s = 1 \Rightarrow C(s) \geq \tau_c \wedge A(s) = 1 \wedge V(s) = 1 \wedge \Pi(s) = \textsc{allow} \wedge s \in \mathcal{F}_{\text{dup}}. \tag{22}$$

Proof sketch. The implication follows by substituting $y_s = 1$ into all five eligibility upper-bound constraints. Because y_s is binary, every upper bound must evaluate to one; otherwise, the corresponding constraint would be violated. Hence, the confidence, approval, validation, policy, and duplicate constraints hold simultaneously.

Proposition 2: Mutation Requires Approval. If $A(s) = 0$, then s cannot be selected for creation under the optimization model:

$$A(s) = 0 \Rightarrow y_s = 0. \tag{23}$$

Proof sketch. Approval condition says $y_s \leq A(s)$. Given $A(s) = 0$ and $y_s \in \{0, 1\}$, the only feasible value is $y_s = 0$. It establishes the human-authority invariant.

The system is set up for use in cases where confidence and approval will be the main factors involved, rather than label associated with the generated output.

3.5 Readiness and Confidence Model

The system calculates readiness and confidence in four steps. It first records which evidence is available. It then calculates a base evidence score, adds a bounded contribution from generated content, and subtracts penalties for missing evidence. Finally, it converts the resulting readiness score into a confidence value.

Let $\mathbf{x} = (x_d, x_a, x_p, x_e, x_h, x_g) \in \{0, 1\}^6$ represent the available evidence for story s:

$$x_d = \mathbb{1}[\text{description exists}], \tag{24}$$

$$x_a = \mathbb{1}[\text{acceptance criteria exist}], \tag{25}$$

$$x_p = \mathbb{1}[\text{dependencies exist}], \tag{26}$$

$$x_e = \mathbb{1}[\text{external evidence available}], \tag{27}$$

$$x_h = \mathbb{1}[\text{activity history available}], \tag{28}$$

$$x_g = \mathbb{1}[\text{edge cases available}]. \tag{29}$$

Each indicator equals one when the corresponding evidence is available and zero when it is missing.

The system uses these indicators to calculate the base evidence score, $R_{\text{src}}(\mathbf{x})$:

$$\begin{aligned} R_{\text{src}}(\mathbf{x}) = {} & 0.25x_d + 0.25x_a + 0.15x_p \\ & + 0.15x_e + 0.10x_h + 0.10x_g. \end{aligned} \tag{30}$$

The weights are positive values and add up to one. Therefore, $R_{\text{src}}(\mathbf{x})$ remains within $[0, 1]$. The description and acceptance criteria have the highest weights since they provide the most crucial information required to evaluate the candidate story.

In create-run mode, the system can apply the bounded bonus B_{gen} for the useful generated content:

$$B_{\text{gen}} = \sum_i w_i z_i, \tag{31}$$

where i indexes quality aspects of the generated text, like the problem statement, acceptance criteria, risk model, or non-functional requirements, and $z_i \in [0, 1]$ is the score for aspect i. Its weight is $w_i \geq 0$, and the sum of the weights satisfies $\sum_i w_i \leq b_{\max}$, where $b_{\max}$ is the maximum permitted generated-content bonus. This upper limit will prevent excessive bonuses being applied to the generated text without supportive evidence.

The system imposes a gap penalty P_{gap} for the important pieces of missing evidence:

$$\begin{aligned} P_{\text{gap}} = {} & \eta_d(1 - x_d) + \eta_a(1 - x_a) \\ & + \eta_p(1 - x_p) + \eta_e(1 - x_e) + \cdots . \end{aligned} \tag{32}$$

Here, the non-negative coefficients η_d, η_a, η_p, and η_e weight the penalties for missing description, acceptance criteria, dependencies, and external evidence, respectively. The rest of the terms will be similar for other missing evidence.

The system calculates the readiness by combining the base score, bonus for the generated content, and penalty for the missing evidence:

$$R(s) = \text{clip}\big(R_{\text{src}}(\mathbf{x}) + B_{\text{gen}} - P_{\text{gap}}, 0, 0.99\big), \tag{33}$$

$$C(s) = \text{clip}\big(R(s) + \delta_C, 0, \gamma_{\text{ceiling}}\big). \tag{34}$$

In this case, the expression $\text{clip}(u, a, b) = \min(\max(u, a), b)$ restricts the input value u to the lower bound a and upper bound b. The expression δ_C represents a bounded confidence modification. The graphical user interface shows the confidence as $C_{10}(s) = 10C(s)$ and truncates it. The evidence-based confidence ceiling $\gamma_{\text{ceiling}} \in [0, 1]$ limits confidence when there is not enough evidence, even if the draft is coherent.

In order to see the effect of evidence on readiness, we can also use the notation $R_s(\mathbf{x})$ for the readiness of story s as a function of its evidence vector. If the bonus and penalty are constant, adding more evidence cannot decrease the readiness measure. Therefore,

$$\mathbf{x}' \geq \mathbf{x} \quad \Rightarrow \quad R_s(\mathbf{x}') \geq R_s(\mathbf{x}), \tag{35}$$

where $\mathbf{x}'$ is a second evidence vector, and $\mathbf{x}' \geq \mathbf{x}$ means that every evidence indicator in $\mathbf{x}'$ is equal to or greater than the corresponding indicator in $\mathbf{x}$.

Lemma 1: confidence is bounded. For any generated candidate s, $R(s) \in [0, 0.99]$ and $C(s) \in [0, \gamma_{\text{ceiling}}]$. *Proof sketch.* Both values are calculated using a clipping operator. By definition, $\text{clip}(u, a, b)$ belongs to $[a, b]$. Thus, $R(s)$ lies in $[0, 0.99]$ and $C(s)$ lies in $[0, \gamma_{\text{ceiling}}]$, irrespective of the raw evidence score, generated bonus, or gap penalty.

Lemma 2: monotonicity under fixed penalties. For $\mathbf{x}', \mathbf{x} \in \{0, 1\}^6$ such that $\mathbf{x}' \geq \mathbf{x}$ componentwise, $R_s(\mathbf{x}') \geq R_s(\mathbf{x})$ if B_{gen}, P_{gap}, and γ_{ceiling} are fixed. *Proof sketch.* The evidence score is calculated as a non-negative weighted sum over $\mathbf{x}$, with all weights being non-negative. Hence, $R_{\text{src}}(\mathbf{x}') \geq R_{\text{src}}(\mathbf{x})$. Applying the same generated bonus and penalty, followed by a monotone clipping function, maintains this inequality.

In practice, the blended formalism shows reviewers how the confidence score was calculated. This allows them to understand whether a particular score is low because of missing acceptance criteria, insufficient evidence, or complex dependencies.

Confidence scope. In this study, confidence is a bounded decision-support score based on the available evidence. It is not a probability of successful delivery. External mutation still requires validation, policy checks, and explicit human approval. Human approval remains the final authorization decision. Therefore, the study does not use confidence to authorize mutation by itself and does not evaluate probabilistic calibration.

Additionally, the model accounts for context-dependent penalties. For example, unresolved architectural dependencies add an uncertainty term to P_{gap} even when the corresponding fields are populated. This approach helps avoid high confidence for tickets that appear complete but remain semantically weak, a concern reported in agile requirements, requirements-smell, and issue-report quality research [9, 15, 36].

3.6 Prompt-Policy Selection Model

The adaptive component in use is a contextual multi-armed bandit [30]. This can be viewed as a restricted short-horizon reinforcement-learning setting, but it does not model a long-horizon Markov decision process or estimate a value function over the workflow state space. We nevertheless refer to it as the *RL layer* because that term is used in the implementation and architecture, while its learning scope remains limited.

3.6.1 Bandit Definition. Each prompt pack p_k is associated with an arm indexed by k in the stochastic multi-armed bandit [5]. The arm's success-probability parameter is θ_k, with Beta success and failure soft counts α_k and β_k. Posterior per arm is:

$$\theta_k \sim \text{Beta}(\alpha_k, \beta_k). \tag{36}$$

At each refinement attempt t, the context state $\mathbf{h}_t$ determines the permissible arm set $\mathcal{K}_{\text{safe}}(\mathbf{h}_t)$. The controller proceeds as follows [45]:

(1) draw a Thompson sample $\widetilde{\theta}_{k,t}$ from the posterior of each arm in $\mathcal{K}_{\text{safe}}(\mathbf{h}_t)$,
(2) choose $k_t = \arg\max_{k \in \mathcal{K}_{\text{safe}}(\mathbf{h}_t)} \widetilde{\theta}_{k,t}$,
(3) apply selected prompt and observe reward proxy,
(4) update $(\alpha_{k_t}, \beta_{k_t})$.

The algorithm for updating the posterior distribution is presented in Algorithm 2.

Eventually, the system will be able to learn what types of prompts are good for which types of issues. For instance, the prompts for technical-debt type of issues will be short, whereas user-facing type issues will require detailed prompts.

Expected performance of arm k:

$$\mathbb{E}[\theta_k] = \frac{\alpha_k}{\alpha_k + \beta_k}. \tag{37}$$

3.6.2 Policy Arms for Prompt Selection. The RL layer works with a limited set of prompt-policy arms. An arm is not just a prompt template, but a bounded policy with the definition of the style of decomposition, evidences, validation strictness, and permissible risks. The separation of arms allows one to control more easily the learning activity, as each arm performs its specific functions and takes certain risks. In Table 2, $\bar{r}_k$ denotes the running mean reward of arm k.

In the process of processing each state of the context $\mathbf{h}_t$, a safety filter eliminates the arms that do not meet the criteria of governance, such as sufficient evidences, allowed workflow state, and writable target. The remaining safe

Table 2. Prompt-policy arms for the RL controller. Arms A–F are examples of arms that are provided to illustrate the selection of arms.

Arm	Strategy	Intended use
A	Conservative decomposition, $\bar{r}_A = 0.61$	Use the safer approach to story crafting in case there is little or high governance risk evidence available.
B	Balanced context strategy, $\bar{r}_B = 0.74$	Consider the coverage, acceptance details, and the reviewer's load if the evidence quality is adequate.
C	Aggressive rewrite, $\bar{r}_C = 0.58$	Focus on the restructuring in case of high ambiguity and reliance on review.
D	Acceptance-criteria enrichment, $\bar{r}_D = 0.5340$	Craft concise and testable acceptance criteria based on Jira summary and available evidence.
E	Dependency-aware planning, $\bar{r}_E = 0.4667$	Start with dependency sequence, blockers, prerequisites, integration requirements, and validation.
F	Evidence-context enrichment, $\bar{r}_F = 0.4875$	Ask focused questions from the reviewer, stay within the scope, and note unknowns in case of sparse evidence.

arms are defined as:

$$\mathcal{K}_{\text{safe}}(\mathbf{h}_t) = \left\{k \in \mathcal{K} \mid \chi_j(k, \mathbf{h}_t) = 1 \text{ for all } j = 1, \ldots, n_\chi\right\}. \tag{38}$$

Here, $\mathcal{K}$ refers to the entire set of prompt arms, while $\mathcal{K}_{\text{safe}}(\mathbf{h}_t)$ refers to the set of arms permissible in attempt t. The state $\mathbf{h}_t$ refers to the contextual state used to decide on the arm. The function $\chi_j(k, \mathbf{h}_t) \in \{0, 1\}$ checks whether the arm k satisfies governance condition j in the context. The output of 1 represents satisfaction of the condition, while output 0 represents rejection of the arm. Here, j represents a governance condition, while n_χ refers to the total number of conditions. Therefore, an arm is included in the safe set only when the values of all conditions are 1.

Following the above filtering process, the controller uses Thompson sampling technique only for the safe arms. For attempt t, the controller draws a value from the Beta posterior of each safe arm and chooses the arm with the highest sampled value:

$$\begin{aligned} \widetilde{\theta}_{k,t} &\sim \text{Beta}(\alpha_k, \beta_k), \qquad k \in \mathcal{K}_{\text{safe}}(\mathbf{h}_t), \\ k_t &= \arg\max_{k \in \mathcal{K}_{\text{safe}}(\mathbf{h}_t)} \widetilde{\theta}_{k,t}. \end{aligned} \tag{39}$$

Therefore, the controller can explore different prompt strategies, but it cannot select an arm that fails a governance condition.

3.6.3 Confidence-Guided Refinement. For each refinement attempt t, the system records the displayed confidence $C_{10,t} \in [0, 10]$. The remaining gap between this confidence and the target $\tau_{10} = 10\tau_c$ on the displayed scale is calculated as:

$$\ell_t = \max(0, \tau_{10} - C_{10,t}), \tag{40}$$

where ℓ_t represents the confidence gap. If the target is not reached, the system uses the latest gap to guide the next refinement attempt. It also stores the complete confidence trajectory for auditing and debugging. The refinement process ends when the confidence reaches the target or when the maximum number of retries is reached. The complete refinement and writeback workflow is shown in Algorithm 1.

3.6.4 Laplace-Domain Convergence View. As the implemented refinement loop operates in a discrete time fashion, its runtime state is encoded in $\{\ell_t\}_{t=1}^{T}$, where T is the attempt horizon. In order to examine its convergence properties we interpolate the lag trajectory in the form of $\ell(t)$ and analyze it in the Laplace domain. Such interpolation serves as

a diagnostic tool, rather than a substitute for the discrete algorithm, and it is used as a cheap way to verify whether confidence lag decays during iterations.

Let the idealized lag dynamics be:

$$\frac{d\ell(t)}{dt} = -\kappa\ell(t) + d_\ell(t), \tag{41}$$

where $\kappa > 0$ represents the convergence rate of the chosen prompt policy and $d_\ell(t)$ denotes a bounded disturbance from lack of evidence, conflicting context, or reviewer feedback. Let s_L denote the Laplace-domain variable, $\Lambda_\ell(s_L) = \mathcal{L}\{\ell(t)\}$, and $D_\ell(s_L) = \mathcal{L}\{d_\ell(t)\}$. The transformed dynamics are:

$$s_L\Lambda_\ell(s_L) - \ell(0) = -\kappa\Lambda_\ell(s_L) + D_\ell(s_L), \tag{42}$$

and therefore:

$$\Lambda_\ell(s_L) = \frac{\ell(0) + D_\ell(s_L)}{s_L + \kappa}. \tag{43}$$

In the no-disturbance case, $D_\ell(s_L) = 0$, so:

$$\Lambda_\ell(s_L) = \frac{\ell(0)}{s_L + \kappa}. \tag{44}$$

The only pole is at $s_L = -\kappa$. Hence the lag dynamics are stable when $\kappa > 0$. By the final value theorem,

$$\lim_{t\to\infty} \ell(t) = \lim_{s_L\to 0} s_L\Lambda_\ell(s_L) = 0, \tag{45}$$

This implies that confidence lag converges to zero when the refinement policy has positive effective gain and no persistent disturbance. If $|d_\ell(t)| \le d_{\max}$, then the steady-state lag magnitude is bounded above by $d_{\max}/\kappa$; hence, larger κ implies faster and tighter convergence.

In practice, the value of κ is estimated from observed attempts using the following formula:

$$\kappa = \max\left(\frac{n}{\sum \delta}, \epsilon\right), \tag{46}$$

where $\epsilon > 0$ prevents zero denominators in calculations. The value of κ can also be recorded along with the confidence trajectory and may provide insight into whether another attempt will help.

3.6.5 Algorithmic Realization. Algorithms 1 and 2 illustrate the described approach in practice.

Algorithm 1 illustrates the full process flow. It involves obtaining context and generating candidate stories. It picks a safe prompt arm and improves each story. A story can be submitted to Jira only after passing the validation, policy, and human review steps.

Algorithm 2 illustrates the prompt-policy update process. It calculates a signed reward based on the change in confidence level, reviewer decision, and write back flag. The reward is mapped to success credit and updated in the selected arm's Beta posterior distribution.

Algorithm 1: End-to-End Governed Epic-to-Story Inference Pipeline

Input: Epic E, connectors $\mathcal{X}$ (Jira/Confluence/Bitbucket), policy state $\mathcal{P}$, creation threshold τ_c, escalation threshold τ_e, max attempts T
Output: Created story set S_{create}, audit trace $\mathcal{T}_{\text{audit}}$

// Retrieve available context, scale the threshold, and initialize the trace.
$\tau_{10} \leftarrow 10\tau_c$; $C \leftarrow \text{RetrieveContext}(E, \mathcal{X})$; $\mathcal{T}_{\text{audit}} \leftarrow \emptyset$
$S_{\text{cand}} \leftarrow \text{GenerateCandidates}(E, C)$
// For each candidate, select a governance-safe arm, enrich, rescore, and validate.
foreach $s \in S_{\text{cand}}$ **do**
  $t \leftarrow 1$; $\ell_0 \leftarrow \tau_{10}$; $A_s \leftarrow 0$
  while $t \le T$ **do**
    $k_t \leftarrow \text{SelectPromptArm}(\mathcal{P}, s, C)$
    $s \leftarrow \text{EnrichWithPrompt}(s, k_t, C)$
    $(R_s, C_s) \leftarrow \text{ScoreReadinessConfidence}(s, C)$
                                                  // $R_s = R(s)$, $C_s = C(s)$
    $C_{10,t} \leftarrow 10C_s$; $\ell_t \leftarrow \max(0, \tau_{10} - C_{10,t})$
    $\widehat{\kappa}_t \leftarrow \text{EstimateLaplaceRate}(\ell_{t-1}, \ell_t)$
                                  // Attempt-level rate estimate; diagnostic only
    $V_s \leftarrow \text{ConsistencyValidate}(s)$
                                                  // $V_s = V(s)$
    $\text{LogAttempt}(\mathcal{T}_{\text{audit}}, E, s, t, k_t, R_s, C_s, \ell_t, \widehat{\kappa}_t, V_s)$
    if $(C_s \ge \tau_c)$ ***and*** $(V_s = 1)$ **then**
      break
    $t \leftarrow t + 1$
  // Determine escalation; policy and explicit human approval remain separate gates.
  // $I_{\text{ctx}}(s)$ indicates limited external context; $I_{\text{esc}}(s)$ indicates required escalation.
  $I_{\text{esc}}(s) \leftarrow \mathbb{1}[C_s < \tau_e] \vee I_{\text{ctx}}(s)$
  $(g_\Pi(s), A_s) \leftarrow \text{PolicyAndHumanGate}(s, I_{\text{esc}}(s))$
                                  // $g_\Pi(s)$ indicates policy permission
  $I_{\text{create}}(s) \leftarrow \mathbb{1}[C_s \ge \tau_c \wedge A_s = 1 \wedge V_s = 1 \wedge g_\Pi(s) = 1 \wedge s \in \mathcal{F}_{\text{dup}}]$
  if $I_{\text{create}}(s) = 1$ **then**
    if *HashCheckAndIdempotencyPass*(s) **then**
                                  // Source and duplicate safety
      $\text{Writeback}(s)$; $\text{AddToCreateSet}(S_{\text{create}}, s)$
return $(S_{\text{create}}, \mathcal{T}_{\text{audit}})$

In Algorithm 2, n_k counts the recorded outcomes for arm k. The values $C_{10,t}^{\text{before}}$ and $C_{10,t}^{\text{after}}$ are its displayed confidence before and after the outcome; d_t is the reviewer decision and w_t is the writeback status. The signed reward $r_t \in [-1, 1]$ is converted to fractional success credit $q_t = (r_t + 1)/2$.

Algorithm 2: Prompt Policy Update (Bandit + Governance-Aware Reward)

Input: Selected-arm state $(\alpha_{k_t}, \beta_{k_t}, n_{k_t}, \bar{r}_{k_t})$ and recorded outcome $(C_{10,t}^{\text{before}}, C_{10,t}^{\text{after}}, d_t, w_t)$
Output: Updated selected-arm state $(\alpha_{k_t}, \beta_{k_t}, n_{k_t}, \bar{r}_{k_t})$
// Build the signed reward from confidence, reviewer decision, and writeback status.
$r_t \leftarrow (C_{10,t}^{\text{after}} - C_{10,t}^{\text{before}})/10$
$r_t \leftarrow r_t + 0.2\mathbb{1}[d_t = \texttt{approve}] - 0.2\mathbb{1}[d_t \in \{\texttt{reject}, \texttt{regenerate}\}]$
$r_t \leftarrow r_t + 0.1\mathbb{1}[w_t = \texttt{written}] - 0.1\mathbb{1}[w_t = \texttt{conflict}]$
// Bound the reward, update the selected arm, and retain audit statistics.
$r_t \leftarrow \text{clip}(r_t, -1, 1)$
$q_t \leftarrow (r_t + 1)/2$
$(\alpha_{k_t}, \beta_{k_t}) \leftarrow (\alpha_{k_t} + q_t, \beta_{k_t} + 1 - q_t)$
$n_{k_t} \leftarrow n_{k_t} + 1$
$\bar{r}_{k_t} \leftarrow \bar{r}_{k_t} + (r_t - \bar{r}_{k_t})/n_{k_t}$
return $(\alpha_{k_t}, \beta_{k_t}, n_{k_t}, \bar{r}_{k_t})$

3.7 Epic-to-Story Decomposition Method

3.7.1 Candidate Generation. By leveraging epic context and previous stories, the system can produce any required capability archetypes:

- UI/interaction story,
- API/contract story,
- persistence/writeback story,
- validation/regression story.

This system rejects the archetypes that are already covered in existing summaries via lexical and semantic matching.

This helps avoid the common error of "story inflation," where assistants create tickets that differ in wording but cover the same work.

In the current implementation, the system recognizes any embedded epic description as a first-class piece of evidence. When an epic is filled with design signals such as solution-design headings, scaling, formulas, APIs and workflows, validation, roll-back considerations, and testing considerations, the system will output a `jira_embedded_context` piece of evidence. This makes the decomposition resilient to the common enterprise challenge where Confluence links are unavailable at the time of execution, while the epic still contains enough design information for bounded planning. In case when the epic has only links and those links are not available, the system avoids the generation of any placeholders and returns the context unavailability message.

3.7.2 Story Point Assignment. Initial estimates are derived from complexity priors:

$$\text{SP}(s) = f(\xi_{\text{scope}}, \xi_{\text{integration}}, \xi_{\text{risk}}, \xi_{\text{unknown}}), \tag{47}$$

with the four ξ factors representing normalized priors for the complexity of scope, integration, risk, and uncertainty. The resulting measure is categorized using Fibonacci-like buckets of 3, 5, and 8. In the current implementation, the default values are deterministic (3/5) and may be learned in future implementations.

Deterministic baseline is an intended choice. It guarantees consistent behavior in initial phases of use and avoids estimation variability until there is enough history to learn from. It also represents a conservative approach to evaluation whereby the value brought in by automation is compared with deterministic baseline performance before applying any other estimation techniques.

3.7.3 Iterative Enrichment. For each candidate story s:

(1) run semantic analysis,
(2) calculate readiness/confidence,
(3) if $C(s) < \tau_c$, provide guidance and repeat the enrichment process within a bounded attempt budget,
(4) provide a structured story payload.

Enrichment increases as each iteration brings explicit feedback from reviewers, such as missed edge cases, unresolved dependencies, or ambiguous non-functional requirements. Thus, the refinement process becomes a collaboration, not just prompting. The distinction is important for successful interaction between human and AI [2, 6].

Moreover, in one embodiment, the output of the enrichment process follows a canonical, strongly typed schema: `problem_statement`, `actor_value`, `functional_scope`, `nfr_constraints`, `acceptance_criteria`, `dependency_map`, `risk_register`, and `test_considerations`. The high degree of typing ensures interoperability of the result with tools for reviews, analytics, and policies and removes ambiguity caused by the informal nature of the free-form text.

An analysis of coverage on the epic level can be done through the capability matrix. Its rows correspond to the capability dimensions such as UI, API, data, observability, resiliency, and security, while its columns represent the suggested stories. The system is able to detect undercovered dimensions and generate candidate stories accordingly. It alters the decomposition process from merely generative to constrained generation with clear objectives.

3.7.4 Ablation Execution Algorithm. Algorithm 3 presents the way how all four modes are implemented for the same epic unit. For each mode, the algorithm activates only the components which fall under this condition, uses the same scorecard, and provides measurements of quality, efficiency, governance, performance, and assessment time.

Algorithm 3: Ablation-Mode Execution for B0/B1/B2/B3 Comparison

Input: Epic unit E_u, mode $m \in \{\text{B0}, \text{B1}, \text{B2}, \text{B3}\}$
Output: Metric row $(Q_m, \text{Eff}_m, G_m, \text{Composite}_m, \text{Time}_m)$
`// Mode schema only; not every evidence block executes every condition.`
if $m = \text{B0}$ **then**
  ExecuteManualReferenceCore(E_u)
    `// Independent manual package required`
else
  ExecuteTemplateCore(E_u) if $m = \text{B1}$
  ExecuteGenericLLMCore(E_u) if $m = \text{B2}$
  ExecuteFullJiraEnhancerCore(E_u) if $m = \text{B3}$
`// Disable B3-only components in baseline modes.`
if $m \neq \text{B3}$ **then**
  Disable components excluded from mode m (critic loop / context packer / validator / governed writeback)
`// Use one scorer; each row retains its measured or analytical status.`
Collect metrics using the same scorer and schema across all modes
Compute Q_m (quality), Eff_m (efficiency), G_m (governance), Composite_m, and Time_m
return $(Q_m, \text{Eff}_m, G_m, \text{Composite}_m, \text{Time}_m)$

3.8 Policy and Governance Layer

The policy and governance layer controls whether a candidate story may move towards Jira creation. It checks confidence, available context, policy rules, validation, and human approval. The AI may generate and refine a story, but it cannot

approve the final writeback. First, the system checks whether a story needs additional human review:

$$I_{\text{esc}}(s) = \mathbb{1}[C(s) < \tau_e] \vee I_{\text{ctx}}(s). \tag{48}$$

where τ_e is the normalized escalation threshold and $I_{\text{ctx}}(s)$ denotes a partial external-context condition. This predicate cannot substitute the explicit approval indicator $A(s)$. Here, $I_{\text{esc}}(s) \in \{0, 1\}$ is the escalation indicator for candidate story s. It equals 1 when the confidence is below τ_e or when external context is incomplete. In this case, the story requires additional human review. Otherwise, it equals 0. Escalation does not replace the mandatory human-approval gate.

The system next checks whether candidate story s is eligible for creation:

$$\begin{aligned} g_\Pi(s) &= \mathbb{1}[\Pi(s) = \textsc{allow}], \\ I_{\text{create}}(s) &= \mathbb{1}[C(s) \geq \tau_c \wedge A(s) = 1 \wedge V(s) = 1 \wedge g_\Pi(s) = 1 \wedge s \in \mathcal{F}_{\text{dup}}], \end{aligned} \tag{49}$$

Here, $\mathbb{1}[\cdot]$ is an indicator function. It returns 1 when a condition is true and 0 otherwise. The term $\Pi(s)$ is the decision made by the active policy. Therefore, $g_\Pi(s) = 1$ means that the policy allows the story.

The creation rule contains five gates. First, $C(s)$ must reach the confidence threshold τ_c. Second, $A(s) = 1$ confirms human approval. Third, $V(s) = 1$ confirms that validation has passed. Fourth, $g_\Pi(s) = 1$ confirms policy permission. Finally, $s \in \mathcal{F}_{\text{dup}}$ confirms that the story is not a duplicate.

The value $I_{\text{create}}(s) = 1$ only when all five gates pass. If any gate fails, the story is not eligible for creation. This value indicates eligibility only. It does not mean that Jira has already been updated. The source-hash and idempotency checks must also pass, as shown in Algorithm 1. The conflict and target checks are applied by the writeback model described below.

Confidence and human approval are important gates. However, neither gate is sufficient by itself. Validation, policy permission, and duplicate control are also required. This design keeps the final decision with a reviewer and supports appropriate reliance on automation [2, 42].

Approval is always attached to a concrete execution run. This way, it can be evaluated in terms of the policy used during the actual execution of the decision, regardless of modifications made to the thresholds afterward. Traceability is important for regulated environments, in which the reproduction of the process is even more important than the outcome itself.

The governance level may also set the approval rules based on the role of the reviewer: changes to the API contract are approved by the domain owner, whereas changes to authentication flow require security approval. Such approval-routing criteria can be represented as policy predicates based on generated metadata and evidence tags.

3.9 Conflict-Safe Writeback Model

The previous section decides whether a candidate may enter the creation stage. It does not make the final change in Jira. In this section, *writeback* means that final Jira change. Approval and writeback may happen at different times. During this gap, the source issue, draft, policy, or target may change. A request may also be retried. The model below blocks stale changes and prevents duplicate creation.

We use two protection levels in this section. A2 combines a source-hash check with atomic idempotency. A3 adds a source-bound transaction envelope. These arrangements are evaluated later in Table 8.

Approval-bound context. An approval flag records the reviewer's decision. However, it does not record the exact context that was approved. A3 therefore stores this context in an approved transaction envelope, denoted by $\mathcal{E}_a$:

$$\mathcal{E}_a = (r_{\text{run}}, i_{\text{issue}}, h_s, v_s, v_d, h_d, v_\Pi, h_\Pi, \mathcal{T}_{\text{target}}, a_{\text{dec}}, u_{\text{rev}}, t_a), \tag{50}$$

where r_{run} identifies the run and i_{issue} identifies the issue. The pair (h_s, v_s) records the approved source hash and revision. The pair (v_d, h_d) records the approved draft version and payload hash. The pair (v_Π, h_Π) records the policy version and snapshot hash. The set $\mathcal{T}_{\text{target}}$ lists the allowed Jira targets. Finally, a_{dec}, u_{rev}, and t_a record the approval result, reviewer, and time.

The system hashes the complete envelope to obtain the transaction identifier $I_{\text{tx}} = H(\mathcal{E}_a)$, where $H(\cdot)$ is a deterministic cryptographic hash function. The A3 idempotency key then binds this transaction to the issue:

$$K_{A3} = H(I_{\text{tx}}, i_{\text{issue}}). \tag{51}$$

Runtime validation. Immediately before writeback, the validator builds the runtime envelope $\mathcal{E}_t$. It has the same fields as $\mathcal{E}_a$. The validator rereads the current source, draft, policy, and target values. It carries forward the run, issue, and approval metadata from the approved envelope.

The system permits a mutation only when the two envelopes match and the decision is still APPROVE:

$$\text{permit}(\mathcal{E}_a, \mathcal{E}_t) = \mathbb{1}[\mathcal{E}_a = \mathcal{E}_t \wedge a_{\text{dec}} = \text{APPROVE}]. \tag{52}$$

An ABA change occurs when the source moves from state A to B and later returns to the same visible content A. Its revision has still changed. The envelope comparison detects this case. It also detects draft regeneration, policy drift, and target drift. A matching Jira text hash alone is therefore not sufficient.

Source-conflict check. The system also calculates a hash of the current normalized issue state:

$$h_{\text{current}} = H(\text{normalized issue state}), \tag{53}$$

It compares this value with the approved source hash $h_{\text{expected}} := h_s$. A mismatch moves the item to the conflict path, and Jira is not changed. Here, $\text{write}(s) = 1$ means that story s is written to Jira. Within the declared validator and fault model, the write authorization rule is:

$$\Pr(\text{write}(s) = 1 \mid h_{\text{current}} \neq h_{\text{expected}}) = 0. \tag{54}$$

Retry protection. While the source check protects from a stale write, repeated requests might cause duplicates. A2 addresses this issue by introducing a payload-level idempotency key:

$$K_{A2} = H(\text{normalized payload},\ \text{target epic},\ \text{policy version}). \tag{55}$$

Requests with the same normalized payload, target epic, and policy version will have the same A2 key. A3 employs the envelope-level key mentioned above. Since $I_{\text{tx}} = H(\mathcal{E}_a)$, its key is equivalently $H(H(\mathcal{E}_a), i_{\text{issue}})$. This binds the retry to the approved context.

The key is verified and reserved atomically before mutation. The hashing alone is not enough since deterministic field ordering makes sure that two logically equivalent payloads have the same hash input. Thus, the request may be treated as the same operation and not a new one. In case the approved state was changed, the story should be revalidated and reapproved. All guarantees hold for the implementation and fault model discussed in Table 8.

Per-story and batch outcomes. The previous rules decide whether one story may be written. A run may have several candidate stories, so the system logs each outcome individually. Let o_s denote the outcome of candidate s, where

$$o_s \in \{\text{CREATED}, \text{ALREADY_EXISTS}, \text{NOT_APPROVED}, \text{CREATE_FAILED}\}.$$

For a candidate batch $\mathcal{B}_{\text{batch}}$, the overall status follows this precedence rule:

$$\text{status}(\mathcal{B}_{\text{batch}}) = \begin{cases} \text{CREATED}, & \exists s \in \mathcal{B}_{\text{batch}} : o_s = \text{CREATED}, \\ \text{FAILED}, & \exists s \in \mathcal{B}_{\text{batch}} : o_s = \text{CREATE_FAILED}, \\ \text{ALREADY_EXISTS}, & \exists s \in \mathcal{B}_{\text{batch}} : o_s = \text{ALREADY_EXISTS}, \\ \text{FAILED}, & \text{otherwise}. \end{cases} \tag{56}$$

The rules are processed from top to bottom. A success in the creation has priority. If nothing has been created, the creation error leads to FAILED. If neither of these conditions is true, the duplicate leads to ALREADY_EXISTS. A batch with only unapproved stories leads to FAILED. The detailed per-story results remain available in every case. The reviewer can therefore see each skipped item, its reason, any connector error, and the existing Jira key for a duplicate.

4 EVALUATION DESIGN

To systematically evaluate the performance, governance capabilities, and human interaction profile of Jira Enhancer, our experimental design maps specific empirical evaluations directly to our four research questions.

4.1 Research Question to Evaluation Mapping

To provide a structured defense of the system, our evaluations are partitioned into four distinct experimental suites:

- **Suite 1 (Field Feasibility - RQ1, RQ2):** Evaluates the system's operational viability using live field telemetry from an industrial deployment tracking professional practitioners interacting with actual requirements.
- **Suite 2 (Independent Quality Audit - RQ1, RQ2):** Assesses the absolute generated artifact quality via a blinded evaluation layout to establish an empirical baseline of narrative fitness.
- **Suite 3 (Controlled Baseline Ablation - RQ4):** Directly compares the generated output of Jira Enhancer against static rule engines and unconstrained model calls to evaluate changes in human assessment duration and required modifications.
- **Suite 4 (Mutation-Safety Fault Injection - RQ3):** Evaluates the technical resilience of our writeback state guards by intentionally introducing transaction anomalies, API drops, and out-of-order execution schedules.

4.2 Experimental Configurations and Model Parameters

The generative treatments used authenticated enterprise model access. The matched human comparison used a common generation configuration for B2 and B3, while varying the number and orchestration of model calls. The later source diagnostic used a separate configuration and is reported separately. Execution records retain the settings associated with each run.

The core contextual prompts used by the system to execute the semantic enrichment loop are standardized across two fundamental operational parameters:

(1) **Decomposition Template:** Instructs the model to extract high-level feature requirements from input text tokens and map them into separate, uniquely identifiable user story summaries without introducing duplicate semantic intents.
(2) **Enrichment Rubric:** Supplies the model with explicit check instructions to evaluate intermediate drafts against eight distinct operational facets, including explicitly measurable acceptance criteria, validation edge cases, and non-functional engineering constraints.

4.3 Human Evaluator Profiles and Recruitment Protocols

To prevent author bias from compromising evaluation integrity, the human rating panels were composed entirely of independent engineering practitioners recruited outside the research group.

- **Field Assessment Panel (Suite 1 & 2):** Comprised two senior software development engineers (SDE-III) holding an average of 8.5 years of industry experience within the enterprise application domain. Both raters were completely decoupled from the tool design and had no prior knowledge of which system configurations produced specific story outputs.
- **Cross-Baseline Comparative Panel (Suite 3):** Comprised three distinct domain experts acting as primary raters, supported by two auxiliary assessors for sensitivity validation. These individuals were active technical product managers and software development managers inside Oracle India Pvt. Ltd., responsible for the exact engineering domains governed by the experimental epic portfolio.

Before launching evaluations, all participants received identical, structured written instructions. Raters were instructed to score each artifact independently using a strictly anchored 1–5 Likert rubric across the prespecified quality dimensions, logging their assessment durations using a programmatically tracked digital workbook interface.

Evaluation of evidence is classified by the way evidence is acquired and the claims it supports. This ensures that field observations, controlled fault injections, and generated sensitivity analyses are not treated as equivalent empirical samples. Table 3 specifies the units of analysis and inference limits for each evidence block.

For reproducibility, the evidence units are organized around story-level endpoints, confidence and readiness summaries, policy decisions, and connector outcomes. The fault-injection units use a fixed random seed and the same transaction schedules for each arrangement. In turn, the generated-unit blocks use their corresponding seeds and transformations. In accordance with empirical software-engineering guidelines on clarity of context, measures, and analysis procedures, we provide the reporting details in this format [26, 44, 49, 57].

Table 3. Evidence hierarchy and claim boundaries. "Generated" denotes deterministic formula-based perturbation or scoring, not an observed execution of the named baseline.

Evidence block	Unit and scale	Primary use	Explicit inference boundary
Authenticated Jira field trace	70 story endpoints; two waves; 14 epics	RQ1 workflow feasibility and RQ2 endpoint telemetry	No randomized control, rejected case, or downstream delivery outcome; independent content ratings are reported separately.
Authenticated-output artifact assessment	Two independent reviewers; 69 full-text stories; 14 epic packages; three masked repeats per reviewer	RQ1 artifact quality and RQ2 reviewability	B3-only absolute assessment; no baseline comparison, downstream outcome, or cross-organization generalization.
Matched blinded baseline assessment	Five reviewers; 42 primary packages (14 inputs × B1/B2/B3); 210 stories; four masked repeats per reviewer	RQ4 package quality, readiness, and reviewer-recorded effort	Reviewers 3–5 form the primary analysis; one portfolio, no manual baseline, no downstream outcome, and no empirical method-origin guess.
Held-out learned-selection diagnostic	Three preselected packets; B2 and B3; two blinded model evaluators; six paired judgments	Exploratory RQ4 text quality and model-evaluator-suggested follow-up prompts	Descriptive only; refinement passes and output length differ; it does not isolate the selection-policy effect.
Controlled mutation-safety ablation	Nine scenarios; 2,000 payloads/scenario; four arrangements; 72,000 method-level observations	RQ3 stale-state, retry, and duplicate handling	Scenario-balanced fault injection; rates are not estimates of production incident prevalence.
Internal generated-unit sensitivity analysis	5,000 formula-derived units per method	Stress-test the declared scoring and composite-index model	B0–B3 metrics are generated; this is not a measured manual-versus-LLM experiment.
Public issue-text sensitivity analysis	300 SWE-bench Lite issue records	Test whether the scoring pipeline accepts public issue text	No patches are generated and no SWE-bench tests are executed; this is not a resolution benchmark result.

5 EMPIRICAL RESULTS AND ANALYSIS

5.1 Field Deployment and Recorded Refinement Effort (RQ1, RQ2)

5.1.1 Authenticated Field Outcomes. Across two deployment waves, the retained Jira records cover 70 story outcomes from 14 epics: 69 new stories and one pre-existing issue identified and linked by duplicate-safe matching. The first wave covered five epics and 25 stories; the second covered nine epics and 45 stories. These records establish that the governed creation and linking workflow executed in the industrial setting.

The latest wave created 44 stories and linked one existing record. All 45 records were attached to their intended parent epic. The 44 newly created payloads each contained an external evidence reference, acceptance criteria, test criteria, and operational-observability criteria. Table 4 summarizes these observed connector and payload outcomes.

Table 4. Authenticated field outcomes and latest-wave payload audit.

Recorded measure	Outcome
Cumulative deployment scope	14 epics; 70 retained story outcomes across two waves
Cumulative creation and reuse	69 newly created stories; one linked existing record
Latest-wave parent linkage	45 of 45 retained records linked to the intended epic
Latest-wave payload audit	44 of 44 new payloads contained all four audited elements

The field confidence pairs were reconstructed after execution from configured defaults and issue properties. They are therefore not used here as native pre-refinement measurements or as a live iteration trajectory. Attempt-level effort is available from the separate matched generation archive, described below.

5.1.2 Recorded Refinement Effort. The matched B3 generation archive retains 159 enrichment attempts for 70 stories across the 14 common epic inputs, in addition to 14 epic-planning calls. Each story had a maximum of three enrichment attempts. The first recorded scores were measured after the first enrichment pass: 68 stories scored 8, one scored 9, and one scored 10. These are recorded post-enrichment scorer outputs; native pre-refinement scores were not archived.

Figure 4 reports the first observed crossing of the internal confidence threshold of 9. Two stories crossed on the first pass, 47 on the second, and eight on the third. The remaining 13 stories stopped below the threshold when their attempt budgets were exhausted and are shown as right-censored. All five stories reached the threshold in four of the 14 epics; ten epics contained at least one below-threshold story at budget end. These descriptive curves measure computational effort to the confidence threshold within the matched artifact-generation study. Human approval and Jira writeback were disabled for this study, so threshold crossing is distinct from an approved mutation.

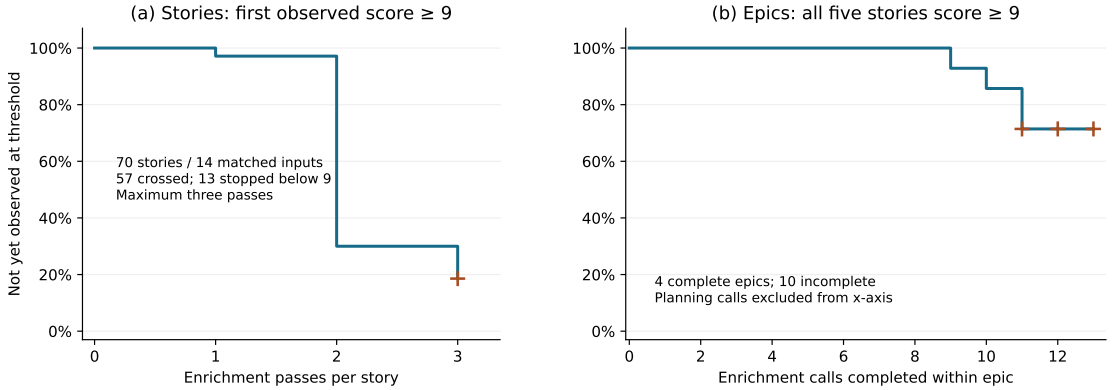


Fig. 4. Recorded refinement effort in matched B3 generation. Panel (a) shows time to the first recorded post-enrichment score of at least 9 for 70 stories. Panel (b) shows enrichment-call effort until all five stories in an epic reach that threshold, excluding the initial planning call. Plus signs mark budget-end censoring. The curves are descriptive product-limit summaries of 14 archived inputs and do not represent the original live deployment or native pre-refinement scores.

5.2 Blinded Independent Artifact Quality Audit (RQ1, RQ2)

RQ1: How can a human-AI workflow split underspecified software epics into reviewable implementation stories without compromising human authority over acceptance and writeback?

RQ2: How can machine confidence, evidence provenance, and policy gates be combined to help reviewers determine whether AI-generated planning records are reliable enough for action?

To assess whether the generated artifacts corresponded to actionable requirements, two independent software-engineering reviewers audited anonymized planning artifacts from the authenticated waves. The review materials omitted method labels, Jira and epic identifiers, internal links, author fields, confidence, readiness, reward, loss, policy-arm values, timestamps, and writeback status. Artifact order was randomized independently for each reviewer. All 70 outcomes remained in scope: 69 stories with full payloads received numerical ratings, while the one pre-existing summary-only record was retained for accounting and marked not rateable. Each reviewer also assessed all 14 five-story

epic packages. Three masked repeats per reviewer were excluded from the primary estimates and retained only for descriptive intra-rater checks.

The reviewers used a prespecified 1–5 anchored rubric, where 3 meant adequate after ordinary clarification or revision. This was a rubric reference point, not an independently validated industry baseline. The story rubric used six dimensions: scope fit, clarity, acceptance/testability, dependency/risk treatment, granularity/actionability, and implementation readiness. The package rubric used coverage breadth, non-redundancy, decomposition coherence, dependency sequencing, and overall readiness. Reviewer scores were first averaged within each artifact. The primary story interval used 50,000 bootstrap resamples of complete epic clusters, preserving the dependence among stories from the same epic.

Table 5. Independent blinded ratings on the prespecified 1–5 scale (69 full-text stories and 14 epic packages).

Measure	Mean
Story composite	3.98
Scope fit	4.97
Clarity	3.90
Acceptance and testability	3.62
Dependency and risk treatment	3.62
Granularity and actionability	4.26
Implementation readiness	3.54
Epic-package composite	4.41
Coverage breadth	4.86
Non-redundancy	4.79
Decomposition coherence	4.75
Dependency sequencing	4.04
Overall package readiness	3.64

The primary analysis of the full-text artifacts returned a mean story composite rating of 3.98/5 (epic-clustered 95% bootstrap CI [3.85, 4.11]). To contextualize this result against the rubric reference value of 3, we applied a two-sided Wilcoxon signed-rank test to the 14 aggregated epic means [55]. All 14 epic means exceeded 3, and the aggregate test was significant ($p = 0.00098$). This supports above-midpoint ratings within the study rubric; it does not establish superiority over an independently executed baseline. The package composite mean was 4.41/5, and all 14 package composites also exceeded 3 ($p = 0.00087$). Neither reviewer recorded an unsupported claim in a rateable story. Scope fit and epic-level coverage were high, whereas acceptance/testability, dependency/risk treatment, and implementation readiness retained more room for reviewer-guided refinement.

Absolute-agreement ICC(2,1) was 0.45 for the story composite and 0.58 for the package composite. Exact agreement across story dimensions ranged from 30.4% to 94.2%, while agreement within one scale point ranged from 91.3% to 100%; every package dimension was within one point in all cases. Some weighted kappa values were near zero despite high observed agreement because variability was low near the top of the scale. We therefore report exact agreement, agreement within one point, and ICC together with kappa rather than relying on kappa alone. Both reviewers reproduced the same composite score for all three masked repeats, although three repeated pairs per reviewer provide only a limited check of consistency.

The reviewers used different cutoffs for categorical decisions. Reviewer 1 labeled 61 stories “Minor revision” and 8 “Major revision” stories, whereas Reviewer 2 labeled 14 stories “Ready,” 48 “Minor revision,” and 7 “Major revision.” We

thus do not aggregate these categories into an overall acceptance rate or state unanimity of readiness. An appropriate interpretation is that the system produced consistent, well-decomposed packages and high-quality artifacts in general, and human review was beneficial for criteria clarification, dependencies, risks, and execution particulars. This block examined only Jira Enhancer outputs; thus, it did not compare the system against manual, template-based, or generic-LLM baselines.

5.3 Cross-Baseline Comparative Analysis (RQ4)

The core baseline evaluation compared the operational characteristics of Jira Enhancer (B3) against a deterministic template rule parser (B1) and a one-shot LLM drafting control (B2). Alongside package quality, the primary evaluation panel (Reviewers 3–5) recorded three outcome parameters: estimated required text modifications, assessment duration, and categorical readiness disposition. These measures assess package review and revision needs, rather than total development efficiency.

RQ4: How does a governed human-AI workflow perform, relative to executable template/rule and generic-LLM baselines, on decomposition quality, review effort, and implementation readiness?

For the matched experiment, we held frozen the same 14 de-identified epic and evidence packages for the three methods. Each method produced five stories per package, according to a common schema. B1 ran the deterministic evidence-section parser and candidate builder. When the parser returned fewer than five stories, a declared static completion rule filled the remaining slots; this occurred for 24 of the 70 B1 stories. B2 and B3 passed through the authenticated OpenAI Codex service using the same model configuration, reasoning effort, and evidence limitation. B2 made one generic generation request per epic, without a critic pass, confidence feedback, policy gate, retries, or reviewer fixes. B3 initially planned the epic and subsequently used Jira Enhancer's story-level enrichment with the bounded critic loop, scoring, and policy validation. Human approval and Jira writeback were off for each method.

Resource consumption is part of the treatment definition. B1 made zero model requests. B2 made 14 successful requests, one for each package, while B3 made 173: 14 planning requests and 159 story-enrichment attempts. Summed model-call durations were 2,087 seconds for B2 and 19,906 seconds for B3. These are cumulative invocation times, not elapsed wall-clock times, because inputs were processed in parallel. This comparison evaluates the complete workflows and is not normalized for model resources.

Five reviewers each received an independently randomized workbook containing 42 primary packages and four variants based on masked repetitions. The variants based on the same input element were separated by at least ten rating rows. All workbooks masked method identity, model and runtime information, source IDs, links, confidence, policy status, and authorship. The reviewers scored, using the anchored 1–5 scale, scope correctness, coverage completeness, story uniqueness, acceptance and testability, dependency and risk handling, granularity and executability, and implementation readiness. They also indicated disposition, confidence, actual time spent on assessment, the estimated number of substantial modifications required, and an evidence-related comment. Because Reviewers 1 and 2 had previously seen B3-only material, the prespecified primary analysis uses Reviewers 3–5. The result from all five reviewers is reported as a sensitivity analysis, and masked repeats are excluded from every method-effect estimate.

The current analysis uses the corrected reviewer ratings and a revised priority-weighted composite. Implementation readiness receives 40%, dependency and risk treatment 20%, and granularity and actionability 20%. Scope fidelity, coverage completeness, story distinctness, and acceptance and testability each receive 5%. These weights total 100% and retain the anchored 1–5 scale. They were selected after the initial analysis to emphasize implementation needs; the weighted outcome is therefore exploratory. The same weights are applied to every method and reviewer.

Table 6. Corrected matched-review ratings for Reviewers 3–5 under the revised priority weights. Weighted scores use the anchored 1–5 scale; time and edit counts are medians [IQR]. Confidence intervals resample the 14 matched epics, and exact sign-flip tests are Holm-adjusted across the two contrasts.

Panel A: Method summaries

Method	Weighted score [95% CI]	Readiness	Time	Edits	Ready/minor
B1	2.56 [2.30, 2.83]	2.50	9.0 [8.0, 10.0]	10.5 [7.0, 14.0]	50.0%
B2	3.59 [3.20, 3.96]	3.52	10.0 [10.0, 11.8]	2.5 [1.0, 5.0]	66.7%
B3	4.32 [4.11, 4.52]	4.60	8.0 [8.0, 9.0]	2.0 [2.0, 3.0]	88.1%

Panel B: Revised weighted-composite contrasts

Contrast	Mean difference [95% CI]	Raw p	Holm p	r_{rb}	+/−/=
B3–B2	0.74 [0.55, 0.93]	0.00012	0.00024	1.00	14/0/0
B3–B1	1.76 [1.43, 2.10]	0.00012	0.00024	1.00	14/0/0

We treated the epic as the unit of inference. Weighted scores were averaged across the three primary reviewers within each input-method cell, leaving 14 matched observations per method. We used a 200,000-draw within-epic permutation test for the three-method comparison; exact paired sign-flip tests for B3–B2 and B3–B1; 100,000-draw epic-bootstrap confidence intervals [13]; Holm correction [18] across the two contrasts within each outcome family; and paired rank-biserial effect sizes. Review time and edit counts used the same matched procedure. The omnibus weighted-composite difference was detectable ($Q_{\text{omni}} = 23.29$, Monte Carlo $p < 10^{-5}$).

Under the applicable IIT Roorkee and Oracle processes, the study owner determined that this internal, low-risk artifact-rating activity did not require separate consent or institutional approval. Reviewers received the study instructions before they returned their completed workbooks. The workbooks used reviewer codes and collected no names, demographic information, sensitive attributes, or other personal identifiers. The de-identified input packets remained access-controlled.

5.3.1 Weighted Requirement Quality and Implementation Readiness. The revised weighted analysis returned a mean score of 3.59 for B2 and 4.32 for B3. The paired mean difference was +0.74 points (95% CI [0.55, 0.93], Holm-adjusted $p = 0.00024$, $r_{rb} = 1.00$). B3 scored higher on all 14 epics. This result supports higher reviewer-rated requirement quality under the declared priority weights. The outcome combines seven requirement dimensions and evaluates the complete workflows.

B3 adds rubric-based gap checks, bounded refinement, confidence tracking, policy evaluation, approval binding, and conflict-safe writeback to the generation process. Candidate drafts can be revised within the attempt budget, and external mutation remains subject to validation, policy checks, and human approval. These mechanisms give the workflow explicit decision and recovery points. The artifact ratings assess the resulting packages; governance and writeback behavior are evaluated separately.

Against B1, B3 was higher on all 14 epics, with a mean weighted difference of 1.76 points [1.43, 2.10] ($p_{\text{Holm}} = 0.00024$, $r_{rb} = 1.00$). The five-reviewer sensitivity analysis gave B3–B2 +0.73 [0.55, 0.92] and B3–B1 +1.78 [1.46, 2.11], with $p_{\text{Holm}} = 0.00024$ and 14 positive epic-level differences for each contrast.

Implementation readiness showed the largest B3–B2 dimension difference: +1.07 points, followed by coverage completeness (+0.86), dependency/risk treatment (+0.79), acceptance/testability (+0.38), granularity/actionability (+0.33), and scope fidelity and story distinctness (+0.21 each). All seven exploratory dimension contrasts were significant after Holm adjustment across the seven dimensions. The adjusted p-value was 0.0059 for readiness, 0.0078 for scope fidelity

and distinctness, and 0.00085 for each remaining dimension. These ratings assess requirement content; the separate transaction tests assess mutation safeguards.

5.3.2 Reviewer Efficiency and Readiness Disposition. B3 packages received Ready-or-Minor-revision ratings in 37 of 42 primary evaluations (88.1%), compared with 28 of 42 for B2 (66.7%). Major-revision ratings were 5 for B3 and 14 for B2. Both methods received 18 Ready ratings; the categorical difference was in the balance between Minor and Major revision. Mean implementation-readiness scores were 4.60 for B3 and 3.52 for B2, a paired difference of 1.07 points (95% CI [0.62, 1.52], Holm-adjusted $p = 0.0059$). The numeric readiness score and categorical disposition are separate recorded outcomes. B1 received Ready-or-Minor-revision ratings in 50.0% of primary evaluations.

Reviewers spent significantly less time assessing B3 packages. Median assessment time was 8.0 minutes for B3 and 10.0 minutes for B2. The paired mean difference was −2.15 minutes (95% CI [−2.35, −1.99], Holm-adjusted $p = 0.00024$, paired rank-biserial $r_{rb} = -1.00$). B3 assessment time was also 0.75 minutes lower than B1 (95% CI [−1.25, −0.28], Holm-adjusted $p = 0.0088$). The recorded duration covers artifact assessment.

The observed time saving is consistent with the intended value of structured refinement, but the experiment did not isolate its cause. Both B2 and B3 used a common output schema and neutral presentation. The difference therefore cannot be attributed to schema enforcement alone. Testing whether gap checks, dependency treatment, or other refinement mechanisms reduce cognitive effort would require a separate comparison and direct workload measures.

Reviewers estimated a median of 2.0 required edits for a B3 package, compared with 2.5 for B2 and 10.5 for B1. The mean required-edit estimate was 0.64 lower for B3 than B2, but the interval included no difference (95% CI [−1.55, 0.19], Holm-adjusted $p = 0.205$). Compared with B1, B3 required 8.02 fewer estimated edits (95% CI [−9.90, −6.21], Holm-adjusted $p = 0.00024$).

For the revised weighted composite, primary-reviewer ICC(2,1) was 0.976, exact pairwise agreement was 66.7%, and agreement within one point was 100%. Among the 12 primary masked-repeat pairs, all 84 dimension ratings and dispositions were identical. Each package received three distinct narrative comments, with mean pairwise text similarity of 0.127. These diagnostics describe agreement in the current corrected ratings and the four repeated items per reviewer.

These findings answer RQ4 within the matched portfolio. Under the revised priority weights, B3 received higher requirement-quality ratings than B1 and B2 on all 14 epics. It also had higher implementation-readiness ratings, lower recorded assessment time, and a more favorable categorical revision profile than B2. The B3–B2 required-edit interval included zero. Policy, approval, recovery, and mutation controls were evaluated separately from the artifact ratings. The weighted outcome is exploratory and concerns the complete tested workflows.

5.4 Held-Out Source Fidelity and Readiness Diagnostic (RQ4)

Our diagnostic examined source fidelity, requirement coverage, and the clarity of implementation criteria in outputs from the learned prompt-selection configuration. It compared one-shot B2 with B3 using governance-filtered arm selection. The model assessments complement the matched human study and are reported separately.

The diagnostic evaluated nine versioned, bounded prompt-policy arms. Each arm represented a refinement strategy. The selected strategy was combined with the current packet evidence and refinement state. One observation for each arm was made on 11 non-held-out training packets. This resulted in 99 critic observations. Learning modified the posterior Beta used for subsequent arm selection. The critic verified four criteria: whether there is a source-grounded gap, the value of the suggested correction, non-redundancy, and grounding accuracy. Three held-out packets did not participate in this process.

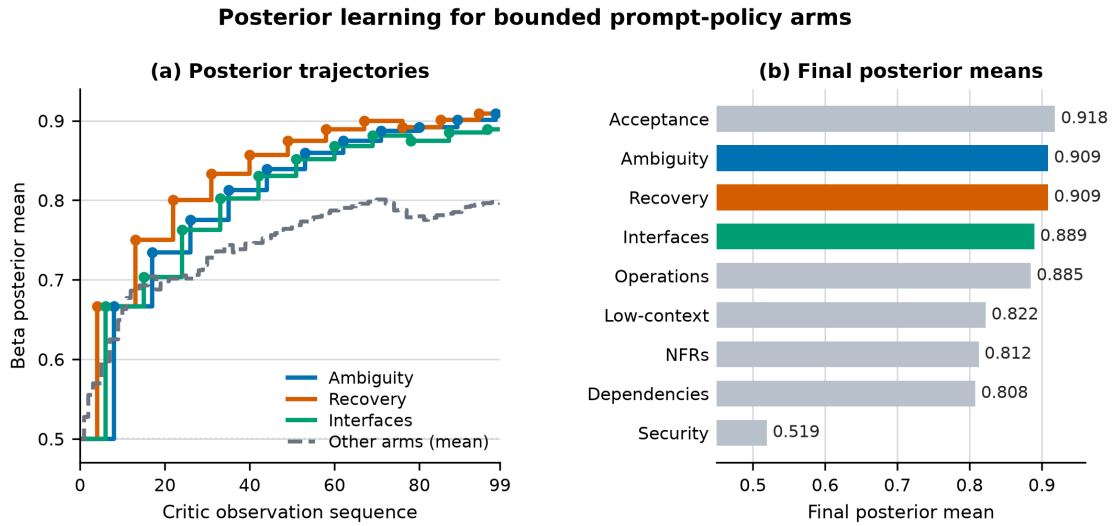


Fig. 5. Learning trace for nine bounded prompt-policy arms in the held-out diagnostic. Panel (a) depicts posterior means of the 99 critic records archived. Three colored lines depict the arms which were selected during exploitation. The gray line depicts the remaining six arms. Panel (b) demonstrates the posterior means of each arm after 11 observations. The held-out packets were not used for learning. The curve describes posterior learning; it is not a curve of human-rated quality or delivery performance.

Each arm started out with a Beta$(1, 1)$ prior. Bounded critic reward $r \in [-1, 1]$ was translated to a fraction of success as $(r + 1)/2$. This value updated the posterior of the respective arm. In each held-out refinement iteration, relevant and governed arms became part of the safe-arm set. The controller picked the best performing arm from the rest of the eligible arms based on posterior mean. Exploration was set to zero, and a selected arm was not reused for the same packet. The controller applied exactly three arms to each packet. Ambiguity resolution, failure and recovery, and interface contracts were selected for all three packets. Their order differed for one packet. Figure 5 shows the recorded posterior trajectory and final arm weights.

The held-out set contained three preselected requirements packets. B2 made one generation pass. B3 made an initial draft and then applied three governed refinement passes. Model configuration and reasoning effort differed between these archived treatments. Two model evaluators received the outputs under masked condition labels and scored eight criteria from 0 to 4.

Both evaluators preferred B3 for every held-out packet. Thus, all six evaluator–packet judgments favored B3. B3 scored 187/192 (97.4%), while B2 scored 148/192 (77.1%), a difference of 39 points or 20.3 percentage points. These percentages describe the available rubric points, not an accuracy rate. B3 scored higher on every criterion in Table 7, including source fidelity (23 versus 20 out of 24), measurable acceptance (24 versus 19), and dependencies and interfaces (24 versus 18).

The source-grounding findings make the difference concrete. Summed across the two evaluators assessing the same three packets, B3 had no unsupported-addition findings, compared with 4 for B2, and no missing-source-requirement findings, compared with 13 for B2. Ambiguous acceptance-criterion findings were 4 for B3 and 21 for B2. These are evaluator-recorded findings, not deduplicated defects or verified production outcomes. They support stronger source

Table 7. Criterion totals in the held-out diagnostic. Each criterion has a maximum score of 24 across three packets and two blinded model evaluators.

Criterion	B2	B3
Source fidelity	20	23
Story boundaries	22	24
Measurable acceptance	19	24
Dependencies and interfaces	18	24
Failure and recovery	18	22
Non-functional and operations	16	22
Evidence and open questions	15	24
Security and compatibility	20	24
Total	148	187

grounding and clearer implementation criteria within this diagnostic. B3 also received 40 model-evaluator-suggested follow-up prompts, compared with 50 for B2.

The diagnostic complements the matched human study with three purposively selected packets and two model evaluators. Its results are descriptive. B3 was 67% longer than B2, and the conditions differed in number of passes and generation settings. The result therefore concerns the complete tested configurations. The matched human comparison supplies separate evidence of higher weighted requirement-quality ratings for B3 under the revised scoring rule.

In Figure 6, the trace of arm learning from the most recent archive is used. There are 99 critic observations from 11 packets that are not held out. The nine different bound strategies have been tested on each of these packets. The heatmap provides information about the reward for each strategy–packet combination. The held-out packets were excluded from learning.

This heatmap exposes the learning signal behind the posterior curves in Figure 5. It is an offline model-critic diagnostic. It is not live attempt telemetry, a human quality rating, or evidence of delivery performance. The posterior trace and held-out selection log are kept separate to avoid training–evaluation leakage.

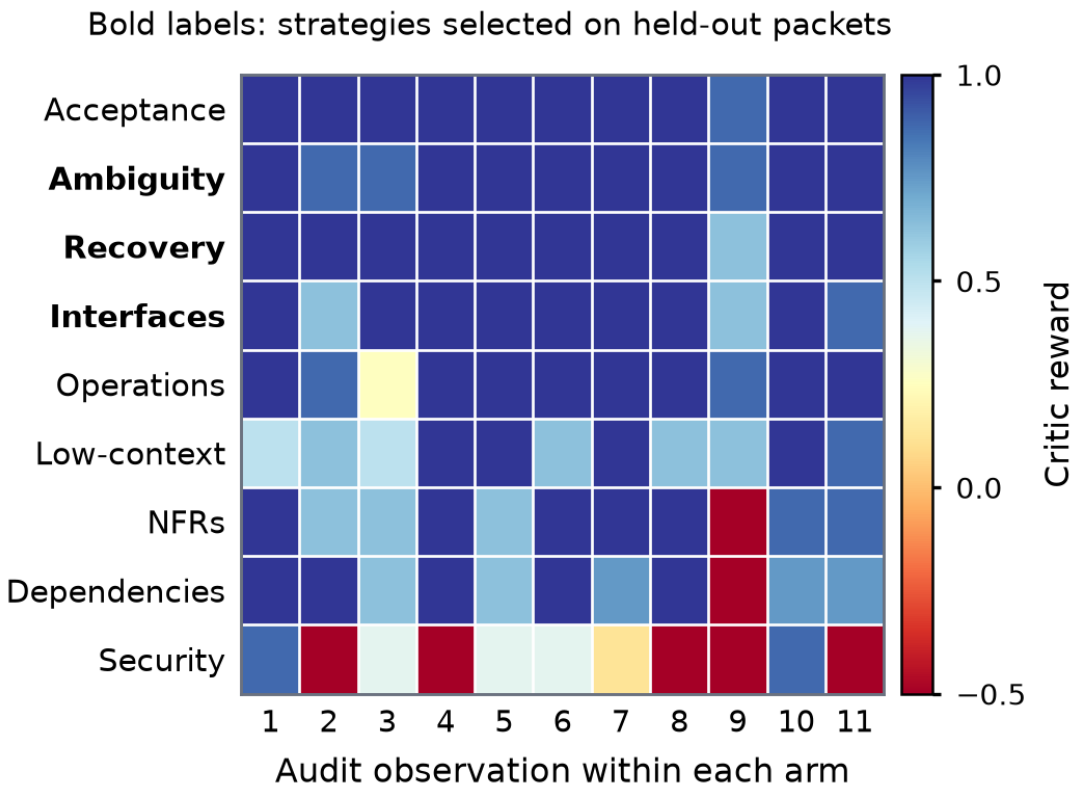


Fig. 6. Recorded rewards from the offline model-critic audit: 99 observations, comprising 11 non-held-out packets for each of nine bounded strategies. Bold labels mark the strategies later selected for held-out refinement. These are model-critic signals, not human quality ratings or delivery outcomes.

5.5 Mutation-Safety Fault Injection Trials (RQ3)

RQ3: What failure modes emerge when agentic planning systems interact with enterprise issue trackers, including missing evidence, duplicate story creation, partial writeback failure, and stale approvals?

We tested the writeback boundary independently of text generation. The experiment applies the same payload and fault schedule to four arrangements: A0 ordinary approval plus logging; A1 source-hash conditional write; A2 source hash plus atomic idempotency; and A3 a source-bound transaction envelope that additionally binds source revision, approved draft version and hash, policy snapshot, reviewer decision, and target set. The nine scenarios are valid control, source edit, source ABA, draft drift, policy drift, target drift, response-loss retry, concurrent retry, and combined source/draft drift. Each scenario contains 2,000 distinct payloads, giving 18,000 transactions for each arrangement and 72,000 method-level observations in total.

Across the equal mixture of nine scenarios, A0 and A1 allowed stale mutations in 66.7% and 44.4% of their respective 18,000 observations. These rates use all method-level observations, rather than only conflict injections. A2 eliminated retry-induced duplicate mutations but retained a 44.4% stale-mutation rate. Only the comprehensive, source-bound transaction envelope architecture (A3) achieved absolute operational isolation, completely eliminating unsafe mutations,

Table 8. Controlled mutation-safety ablation. Rates use an equal mixture of the nine declared scenarios and therefore measure coverage under the injected schedule, not production prevalence. Eligible-write success is computed over valid-control and retry scenarios.

Arrangement	Stale mutation	Duplicate mutation	Any unsafe mutation	Correct outcome	Eligible-write success
A0 ordinary approval/log	66.7%	33.3%	88.9%	11.1%	33.3%
A1 source-hash conditional	44.4%	22.2%	66.7%	33.3%	33.3%
A2 source hash + atomic idempotency	44.4%	0.0%	44.4%	55.6%	100.0%
A3 source-bound transaction envelope	0.0%	0.0%	0.0%	100.0%	100.0%

duplicate tickets, and stale authorization blocks (0.0% mutation rate) across all 72,000 automated injection schedules. The denominator for A3 itself is 18,000 observations; 72,000 is the total across A0–A3. Here, the 0.0% rate refers to unsafe mutations, not to all writes. Stale approvals were blocked, while 100% of eligible writes were permitted. The result is bounded by the declared implementation and fault model.

A3 blocked every injected stale mutation and condensed retry duplicates into a single remote mutation while preserving every eligible write. Comparison with A2 highlights the impact of linking approval with factors other than the current status of the source. Using the hash of only the source does not help in detecting ABA modifications, generation of new drafts, changes in policy, or changes in the target set, in case there is no change in the Jira text. Seven test cases illustrate proper envelope construction, protection against stale approvals, and allowing of idempotence. These test cases highlight how the protection of stale approvals and idempotence happens considering the fault model defined. The frequency and the impact of these measures are not evaluated as part of the production usage.

Overall, the live block demonstrates that the authenticated workflow is feasible and gives the pairs of endpoint telemetry.

6 DISCUSSION AND WORKFLOW TRADE-OFFS

Our empirical findings provide a bounded answer to the research questions and expose the trade-offs in applying agentic systems to shared engineering records. The central insight is that text fluency must not be conflated with requirement readiness. A single-turn language model can produce useful requirement drafts, but the studied B2 control ends at generation. It does not implement the evidence, approval, recovery, and writeback safeguards provided by B3.

6.1 Operational Advantages of B3 over One-Shot Drafting

The utility of a requirement-generation system includes source grounding, implementation readiness, and control over tracker changes. The evaluations identify three advantages of the complete B3 configuration:

(1) **Source fidelity and omission findings.** In the held-out diagnostic (Section 5.4), two blinded model evaluators assessed the same three packets. Their summed findings included 4 unsupported additions and 13 missing source requirements for B2, compared with zero of each for B3. Ambiguous acceptance-criterion findings were 21 for B2 and 4 for B3. Both evaluators preferred B3 on every packet, with combined scores of 187/192 for B3 and 148/192 for B2. These descriptive assessments support stronger source grounding within the diagnostic.

Table 9. Comparison of failure-mode coverage. This table presents a mechanism-oriented analysis derived from implemented behavior, not a measured frequency table. It identifies the collaboration failures explicitly represented and recoverable in each approach.

Failure mode	B0 manual	B1 template/rule	B2 generic LLM	B3 Jira Enhancer
Missing external evidence	Manual delay or meeting follow-up	No evidence-aware handling	Risk of fluent but weakly grounded output	Warning codes plus embedded Jira-context fallback when substantive design text exists
Duplicate story creation	Depends on reviewer memory	Weak summary matching only	Possible if generated wording changes	Deterministic normalized-summary skip with existing issue key reported
Partial create failure	Manual reconciliation	No preserved decision state	No structured partial outcome	Per-story `create_failed` status, skipped list, and retry-preserved UI state
Stale approval after tracked edit	Human process risk	Not modeled	Not modeled	Source-hash check blocks writeback and forces revalidation
Unsafe high-confidence output	Reviewer judgment only	Static guardrails	Confidence may be uncalibrated	Confidence threshold, validation, policy gate, and explicit approval key

They are not human-panel ratings, deduplicated defect counts, or a guarantee that the verification loop will detect every semantic error.

(2) **Protection against stale and duplicate mutations.** B3's writeback mechanism binds approval to the reviewed source revision, draft, policy, and target set. In Table 8, the A1 and A2 source-hash arrangements each retained a 44.4% stale-mutation rate across their scenario-balanced observations. A3 recorded zero unsafe mutations in its 18,000 observations and permitted all eligible writes. The full A0–A3 suite contains 72,000 observations. These are tests of transaction mechanisms, not evidence that the text-only B2 condition itself caused production failures; Jira writeback was disabled in the matched artifact comparison. The result supports the source-bound execution barrier under the declared fault model.

(3) **Higher weighted requirement quality and implementation readiness.** B3 scored 4.32 versus 3.59 for B2 under the revised priority weights, a paired difference of 0.74 points ($p_{\text{Holm}} = 0.00024$). Mean implementation-readiness ratings were 4.60 versus 3.52. Ready-or-Minor-revision ratings were 88.1% versus 66.7%, with 5 versus 14 Major-revision ratings; both methods received 18 fully Ready ratings. B3 also reduced recorded assessment time by 2.15 minutes ($p_{\text{Holm}} = 0.00024$). These outcomes describe the corrected ratings in the matched portfolio and the complete tested workflows.

Table 9 identifies the implemented controls behind these workflow differences. It is a mechanism-coverage comparison, not a production failure-frequency estimate. B0 was not executed on the matched inputs; its lower scores occur only in the formula-based stress test, so no measured B3–B0 superiority claim is made.

6.2 The Computational Resource Trade-off

The operational benefits require additional model work. B2 made 14 model calls, with a cumulative invocation duration of 2,087 seconds. B3 made 173 calls, with a cumulative duration of 19,906 seconds. These totals sum model-call durations; they are not elapsed wall-clock times because inputs were processed in parallel. The comparison therefore does not establish computational efficiency for B3.

The extra model work may be worthwhile where review effort and safe tracker updates are important, but that deployment decision needs measured costs. The study did not record inference charges, financial losses from tracker errors, or downstream delivery savings. It therefore cannot establish that backend costs are negligible or that the trade-off is financially favorable in every organization. The observed trade-off is additional model work alongside higher weighted requirement-quality ratings, lower assessment time, a more favorable revision profile, and the tested mutation safeguards.

6.3 Answers to the Research Questions

RQ1: Governed decomposition. The field telemetry and quality audits show that the workflow can decompose corporate epics into reviewable stories while preserving a separate human decision before writeback. Across two deployment waves, 69 stories were created and one existing record was linked through duplicate-safe matching. This establishes successful-path feasibility within the evaluated setting.

RQ2: Confidence, evidence, and readiness. Machine confidence, evidence provenance, and policy checks give reviewers explicit reasons to accept, revise, or reject a candidate. Confidence is optimized by the pipeline itself, so reaching the threshold is not independent evidence of correctness. The source-grounding findings in the model diagnostic and the human-rated revision profile provide separate assessments of requirement quality; human approval remains necessary for domain decisions and external mutation.

RQ3: Tracker failure modes. The fault-injection suite identifies stale-source, draft, policy, and target changes, together with lost-response and concurrent retries. Under the tested fault schedules, simpler controls permitted stale or duplicate mutations. A3 recorded zero unsafe mutations in its 18,000 observations and allowed every eligible write. Human authorization therefore needs to remain tied to the specific source, draft, policy, and target state reviewed by the user. The experiment tests this execution barrier, not the production frequency of faults.

RQ4: Quality and workflow trade-offs. In the revised weighted analysis, B3 exceeded B2 by 0.74 points and B1 by 1.76 points, with higher scores on all 14 matched epics and Holm-adjusted $p = 0.00024$ for each contrast. B3 also received higher implementation-readiness ratings, a more favorable revision profile, and a 2.15-minute reduction in recorded assessment time relative to B2. The separate held-out model diagnostic favored B3 on source fidelity, coverage, and implementation criteria. The estimated-edit contrast with B2 remained uncertain (95% CI $[-1.55, 0.19]$, $p = 0.205$). These findings concern the complete workflows and the revised exploratory outcome. B3 used more model resources.

6.4 Reliability and Recovery Implications

The fault results show why retry handling must preserve the approved decision and distinguish a repeated request from a new mutation. The workflow in Section 3.1 keeps selected stories and partial outcomes visible so that a reviewer can inspect, retry, revise, or cancel. Reduced-context warnings also make missing evidence visible instead of treating a fluent draft as fully supported. These controls explain how the system supports recovery; their effects on user behavior and production incident rates require further study.

6.5 Positioning and Novelty Scope

Evidence blocks can validate various claims: 70 authenticated endpoints show that a successful path is feasible; independent ratings were provided for 69 full-text stories and 14 epic packages; five experts compared 42 B1, B2, and B3 packages together with 210 stories; fault injection shows safety of writeback; 5,000-unit and public issue text blocks show declared numerical behavior. The study does not measure cross-project and longitudinal delivery benefits, which remain important in light of unreliability, human values, and unavailability of standardized evaluation of AI in software engineering [1, 12, 14].

The implementation includes configurable prompt-arm selection. B3 does not use an unrestricted prompt loop. It organizes refinement strategies as versioned, bounded policy arms. At runtime, the selected strategy is combined with the current evidence, gap signals, confidence, and attempt state. A contextual bandit selects only from arms that pass the governance checks [5, 30]. Learning updates the Beta posterior used for later selection. The selected arm, refinement result, and posterior update are recorded for auditing, as shown in Table 2 and Algorithm 2. This design supports adaptive selection without giving the learning layer permission to modify Jira. Final mutation still requires validation, policy checks, and human approval.

The matched human study used Thompson sampling over a frozen pre-study posterior as a controlled evaluation configuration. Posterior updates were disabled during artifact generation to prevent reviewer outcomes from influencing later artifacts. Prompt-arm selection, bounded refinement, critic checks, and policy evaluation remained active. Freezing the posterior therefore did not turn off arm selection or eliminate sampling variability. This was a comparison of complete workflows, not a component ablation: B2 and B3 used a common output schema, and the experiment did not isolate the schema, critic, or learning policy's contribution.

The subsequent diagnostic (Section 5.4) separately recorded 99 offline critic observations across 11 non-held-out packets, excluding the three evaluation packets. Figure 5 shows the posterior updates; the held-out stage then used governance-filtered, zero-exploration selection. These records make learning and selection auditable, but they do not establish statistical convergence or isolate the benefit of learning. Together, the two evaluations support workflow-level findings and inspection of the bandit mechanism. They do not measure live human interaction with an updating policy. A matched comparison against fixed or rule-based arm selection is needed to estimate learning's incremental value.

B3 controls learning, confidence, governance, approval, recovery, and conflict-free writeback. The score of B3's blinded artifact did not differ significantly from B2's, but B3 took less assessment time, had a descriptively better revision profile, and was the only one that could proceed through a source-bound mutation pathway tested under stale state, retry, draft drift, policy drift, and target drift. This evidence supports the bounded advantage of end-to-end workflows.

7 LESSONS LEARNED

The study produced six practical lessons. Each lesson is limited to the evidence that supports it.

(1) **Different evaluations answer different questions.** The study uses four types of evidence, as summarized in Table 3. First, the field trace contains 70 verified end states. These include 69 newly created Jira issues and one safe-duplicate resolution. The latest wave also includes 44 audited new payloads. Second, human reviewers assessed artifact quality. Two reviewers assessed 69 full-text stories and 14 epic packages. Five reviewers compared 42 matched B1, B2, and B3 packages containing 210 stories. Third, the fault-injection experiment tested mutation safety across 72,000 scenario-balanced observations. Finally, formula-based sensitivity analyses

remain diagnostic evidence. These evaluations have different goals and units. Therefore, their results are reported separately and are not combined into one overall result.

(2) **B3 automates refinement that B2 leaves outside the pipeline.** B2 generated one generic version per epic and then terminated. There was no critic step, confidence feedback, retry process, or prompt policy choice. Further refinement needed human guidance or another controller. B3 employed a bounded critic loop with learned, governance-filtered prompt arm selection. In the parallel experiment, the average assessment time was 2.15 minutes shorter with B3. The reviewers made 5 Major revision choices for B3 and 14 for B2. The revised weighted requirement-quality score was 0.74 points higher for B3 (Holm-adjusted $p = 0.00024$). In the subsequent held-out diagnostic test, the model assessors proposed 40 further prompts for B3 and 50 for B2. They also preferred B3 on all six comparison judgments. These data favor automatic, governed refinement with less follow-up. They do not indicate better prose or reduced compute cost overall. Human judgment is required for omitted domain choices and final approval.

(3) **Human approval requires a technical limit.** Simply approving the draft cannot prevent the mutation from being stale or redundant. The approved draft must stay tied to the source state that was reviewed by the human. This is accomplished through the source-tied transaction envelope. During the fault injection experiment, it prevented all injected dangerous mutations and permitted all legitimate writes, as noted in Table 8. Human power is greater when the writeback contract enforces the approval decision.

(4) **Missing evidence must be made known.** The system can proceed in reduced context when there is no external evidence. It cannot hide the lack of source evidence. The warning codes and confidence provenance will assist the human in understanding this problem. The draft can still be valuable without being complete evidence.

(5) **The recovery process should maintain human control.** The generation and write-back procedures can fail at any stage. The workflow should enable the reviewer to continue, try again, or cancel. Regeneration without human interaction would compromise control and auditing.

(6) **Adaptive models require full trajectories and cost proof.** The live data have the terminal states but not each of the refinement attempts. They cannot prove a complete live learning trajectory. In the latter, there were 99 observations from an offline critic. Those observations revealed how the posterior probabilities evolved but they were model-generated data and not human and delivery outcomes. In comparison to B2, B3 did more model work in both comparisons. Hence faster package assessment does not mean lower cost of computation. Future studies should have each attempt recorded and the outcomes normalized with respect to cost.

Collectively, these lessons lead to a modest conclusion. B3 shows an established path from data to an approved mutation based on the analyzed conditions. The paper does not demonstrate a better prose, delivery quality, productivity, computing efficiency or generalization over different projects.

8 IMPLICATIONS AND THREATS TO VALIDITY

Our findings carry practical implications for engineering organizations: generative AI assistants should be deployed as governed workflows, with explicit review and controlled tracker updates. Teams should assess source fidelity, missing requirements, acceptance-criteria clarity, and revision readiness separately from prose fluency and the system's internal confidence score. A draft can be useful while still needing domain clarification. Before any external write, approval should remain bound to the reviewed state, and recovery should preserve the reviewer's decisions.

The evaluations also expose a resource trade-off. The matched B3 workflow used 173 model calls, compared with 14 for B2. Lower human assessment time therefore does not establish lower total computational cost. Organizations should measure both model resources and subsequent engineering outcomes when deciding where governed refinement is useful.

8.1 Study Scope and Validity

Following the validity classification described by Wohlin et al. [57], we assess internal, construct, external, and conclusion validity. **Internal validity:** this concerns whether the observed differences come from the evaluated methods rather than the study design. We used the same inputs, neutral formatting, independent randomization, and blinding to reduce bias [57]. However, the matched workflows were not compute-matched. B2 used 14 model calls, while B3 used 173. For B1, the static completion rule filled slots for 24 of 70 stories. Reviewers 1 and 2 had previously seen B3-only material. We therefore excluded them from the primary analysis. The later held-out diagnostic also compared unequal configurations. B2 used one pass, while B3 used three refinement passes after its initial draft. Their model configurations and reasoning settings also differed. B3 was 67% longer. The diagnostic therefore cannot isolate the effect of learned selection. The generated-unit analysis also uses formulas rather than executed treatments. It cannot estimate a treatment effect [49].

Construct validity: this concerns whether the measures represent the concepts claimed in the study. Confidence is produced and optimized by the same pipeline. It is therefore not an independent measure of quality. Human approval records a workflow decision, not later delivery success. Blinded reviewer ratings measure visible content quality. Recorded assessment time is not total planning time, and the edit counts are estimates. Reviewers could not see the governance metadata. Therefore, the artifact score does not measure policy or writeback value.

Human Factors and Interactive Limits. Because our evaluation design prioritizes artifact-level actionability, writeback transaction integrity, and total manual review duration, it does not explicitly log standard human-computer interaction (HCI) parameters such as NASA-TLX cognitive workload indices, System Usability Scale (SUS) scores, or real-time user fatigue variables over multi-sprint intervals. While our efficiency data confirm a significant 2.15-minute reduction in assessment time, this metric tracks out-of-band validation behavior rather than real-time interaction patterns inside a live console interface. Consequently, our findings establish lower requirement evaluation friction without claiming optimal interface layout ergonomics, calibrated user trust balance, or long-horizon psychological adoption rates.

External validity: this concerns whether the findings can be applied to other settings. The 70 endpoints come from 14 epics, two deployment waves, and one organization. They represent successful paths only. The data contain no rejected, abandoned, below-threshold, or conflict endpoint. The matched study uses the same portfolio and has no independently authored manual baseline. The learned-selection diagnostic uses only three purposively selected packets and two model evaluators. The public issue-text analysis also does not execute the methods. Claims about transfer therefore require new projects, domains, and practitioners. This is especially important for context-dependent socio-technical tools [49, 50].

Conclusion validity: this concerns the strength and precision of the conclusions. The epic-level exact and cluster-resampled analyses use 14 matched epic clusters. The current analysis uses corrected reviewer ratings and weights chosen after the initial analysis, so the weighted findings are exploratory. Holm adjustment covers the declared contrast families; independent replication with the weights fixed in advance would assess their stability on new inputs. Reviewer agreement and the four repeated items per reviewer describe consistency in this portfolio. The held-out diagnostic has six paired model judgments and is descriptive, as are the disposition counts. Fault-injection rates describe the tested scenario mixture.

Reference currency: research on human–AI collaboration in software engineering is developing quickly. The related literature should therefore be reviewed and updated periodically [56].

Mitigation: future studies should use independently prepared manual packages and compute-matched B2 and B3 configurations. The blinded study should be repeated on new portfolios, and reviewer expertise levels should be recorded. Ablation tests should disable the critic, retrieval, validation, and policy layers one at a time. Future work should also test sensitivity to the selected weights and assess the practical significance of any change. Finally, later studies should observe edits, failures, rework, and cycle-time variation across multiple releases.

8.2 Artifact and Confidentiality Boundary

The review artifact retains deterministic analysis scripts, de-identified telemetry for the 70 story endpoints, a de-identified audit of the latest-wave payloads, coded ratings from both artifact assessments, aggregate matched-test tables, fault-injection trial records, seeds, and checksums. The shareable data exclude issue and epic keys, internal URLs, account names, email addresses, and timestamps. Raw Jira and Confluence records, the sanitized matched-input packets, and the private method-allocation key remain access-controlled because the source requirements are confidential enterprise material. We did not collect reviewer names, demographic variables, sensitive attributes, or other personal identifiers. The study also does not measure trust, workload, satisfaction, or individual performance.

8.3 Future Extensions

Further research may advance the work of Jira Enhancer in five dimensions. First, multi-objective reinforcement learning can find a trade-off between the confidence level and the cost of models. Second, project velocity will allow story-point estimation. Third, active learning can discover the stories that should be manually verified. Fourth, multilingual prompts will enable the work of the teams using different languages. Finally, dependency graph will link Jira issues, documents, code, and incidents. This graph will help the system to locate the related work and critical paths before decomposition of an epic.

9 CONCLUSION

This paper addresses a critical challenge in agentic software engineering: how to leverage generative AI to decompose incomplete corporate epics into execution-ready stories without sacrificing tracking control, accountability, or data integrity. We presented Jira Enhancer, an operational framework that wraps generative model calls inside programmatic policy gates, machine confidence metrics, and conflict-safe writeback transactions.

The industrial deployment involved a team of 40 professional practitioners across 14 epics and established successful-path feasibility. The main quality findings concern source grounding and revision readiness. In the three-packet held-out diagnostic, two model evaluators recorded no unsupported additions or missing requirements for B3, compared with 4 and 13 findings for B2; ambiguous acceptance-criterion findings were 4 versus 21. B3 scored 187/192 versus 148/192, and both evaluators preferred it on every packet. These are descriptive model assessments, not an independently verified accuracy rate or an isolated estimate of the benefit of learning.

In the matched human study, the revised weighted analysis gave B3 a mean score of 4.32, compared with 3.59 for B2 and 2.56 for B1. B3 scored higher than both baselines on all 14 epics. The B3–B2 paired difference was 0.74 points (95% CI [0.55, 0.93], Holm-adjusted $p = 0.00024$). B3 received 88.1% Ready-or-Minor-revision ratings compared with 66.7% for B2, and 5 Major-revision ratings compared with 14. Both received 18 fully Ready ratings. B3 assessment time was

2.15 minutes lower, while the required-edit interval included zero. The weights were revised after the initial analysis, and the reported comparison is exploratory. B3 used additional model resources to provide these workflow capabilities.

The source-bound transaction envelope recorded zero unsafe mutations and permitted all eligible writes in A3's 18,000 fault-test observations. This result is bounded by the tested implementation and fault schedules. The practical takeaway is to combine source-grounded refinement and visible revision needs with an enforceable approval boundary; fluent text or high internal confidence alone is insufficient authorization for a tracker update. Broader claims about delivery quality, productivity, or cross-project transfer require further evaluation.

To support replication, the implementation, analysis scripts, and shareable evidence are available at https://github.com/manish94u/JiraEnhancer. The repository includes de-identified field tables and aggregate evaluation records. Confidential source requirements and restricted records remain subject to the access boundary described in Section 8.2.

REFERENCES

[1] Iftekhar Ahmed, Aldeida Aleti, Haipeng Cai, Alexander Chatzigeorgiou, Pinjia He, Xing Hu, Mauro Pezzè, Denys Poshyvanyk, and Xin Xia. 2025. Artificial Intelligence for Software Engineering: The Journey So Far and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5, Article 119 (2025), 27 pages. https://doi.org/10.1145/3719006

[2] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. 2019. Guidelines for Human-AI Interaction. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 1–13.

[3] John Anvik, Lyndon Hiew, and Gail C. Murphy. 2006. Who Should Fix This Bug?. In *Proceedings of the International Conference on Software Engineering*. Association for Computing Machinery, New York, NY, USA, 361–370.

[4] Akari Asai, Sewon Min, Zexuan Zhong, Danqi Chen, Hannaneh Hajishirzi, Wen-tau Yih, Yejin Choi, Mike Lewis, and Luke Zettlemoyer. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Vienna, Austria.

[5] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-Time Analysis of the Multiarmed Bandit Problem. *Machine Learning* 47, 2–3 (2002), 235–256. https://doi.org/10.1023/A:1013689704352

[6] Gagan Bansal, Besmira Nushi, Ece Kamar, Walter S. Lasecki, Daniel S. Weld, and Eric Horvitz. 2019. Beyond Accuracy: The Role of Mental Models in Human-AI Team Performance. In *Proceedings of the AAAI Conference on Human Computation and Crowdsourcing*. AAAI Press, Palo Alto, CA, USA, 2–11.

[7] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111.

[8] Philip A. Bernstein and Eric Newcomer. 2009. *Principles of Transaction Processing* (2 ed.). Morgan Kaufmann, Burlington, MA, USA.

[9] Nicolas Bettenburg, Sascha Just, Adrian Schröter, Cathrin Weiss, Rahul Premraj, and Thomas Zimmermann. 2008. What Makes a Good Bug Report?. In *Proceedings of the ACM SIGSOFT International Symposium on Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 308–318.

[10] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. Curran Associates, Inc., Red Hook, NY, USA, 1877–1901.

[11] Lan Cao and Balasubramaniam Ramesh. 2008. Agile Requirements Engineering Practices: An Empirical Study. *IEEE Software* 25, 1 (2008), 60–67.

[12] Haowei Cheng, Jati H. Husen, Yijun Lu, Teeradaj Racharak, Nobukazu Yoshioka, Naoyasu Ubayashi, and Hironori Washizaki. 2026. Generative AI for Requirements Engineering: A Systematic Literature Review. *Software: Practice and Experience* 56, 2 (2026), 141–170. https://doi.org/10.1002/spe.70029

[13] Bradley Efron. 1979. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics* 7, 1 (1979), 1–26. https://doi.org/10.1214/aos/1176344552

[14] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, Melbourne, Australia, 31–53. https://doi.org/10.1109/ICSE-FoSE59343.2023.00008

[15] Henning Femmer, Daniel Méndez Fernández, Stefan Wagner, and Sebastian Eder. 2017. Rapid Quality Assurance with Requirements Smells. *Journal of Systems and Software* 123 (2017), 190–213.

[16] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *The Twelfth International Conference on Learning Representations*. OpenReview.net, Vienna, Austria.

https://proceedings.iclr.cc/paper_files/paper/2024/hash/fef126561bbf9d4467dbb8d27334b8fe-Abstract-Conference.html

[17] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: Retrieval-Augmented Language Model Pre-Training. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, Virtual, 3929–3938. https://proceedings.mlr.press/v119/guu20a.html

[18] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70. https://www.jstor.org/stable/4615733

[19] Irum Inayat, Siti Salwah Salim, Sabrina Marczak, Maya Daneva, and Shahaboddin Shamshirband. 2015. A Systematic Literature Review on Agile Requirements Engineering Practices and Challenges. *Computers in Human Behavior* 51 (2015), 915–929.

[20] ISO/IEC/IEEE. 2018. *ISO/IEC/IEEE 29148:2018 Systems and Software Engineering—Life Cycle Processes—Requirements Engineering*. ISO/IEC/IEEE.

[21] Gaeul Jeong, Sunghun Kim, and Thomas Zimmermann. 2009. Improving Bug Triage with Bug Tossing Graphs. In *Proceedings of the Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 111–120.

[22] Zhengbao Jiang, Jun Araki, Haibo Ding, and Graham Neubig. 2021. How Can We Know When Language Models Know? On the Calibration of Language Models for Question Answering. *Transactions of the Association for Computational Linguistics* 9 (2021), 962–977. https://doi.org/10.1162/tacl_a_00407

[23] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Vienna, Austria.

[24] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Online, 6769–6781. https://doi.org/10.18653/v1/2020.emnlp-main.550

[25] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. Association for Computing Machinery, New York, NY, USA, 39–48. https://doi.org/10.1145/3397271.3401075

[26] Barbara A. Kitchenham, Shari Lawrence Pfleeger, Lesley M. Pickard, Peter W. Jones, David C. Hoaglin, Khaled El Emam, and Jarrett Rosenberg. 2002. Preliminary Guidelines for Empirical Research in Software Engineering. *IEEE Transactions on Software Engineering* 28, 8 (2002), 721–734.

[27] Andrew J. Ko, Brad A. Myers, and Duen Horng Chau. 2006. A Linguistic Analysis of How People Describe Software Problems. In *Proceedings of the IEEE Symposium on Visual Languages and Human-Centric Computing*. IEEE Computer Society, Los Alamitos, CA, USA, 127–134.

[28] John D. Lee and Katrina A. See. 2004. Trust in Automation: Designing for Appropriate Reliance. *Human Factors* 46, 1 (2004), 50–80.

[29] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. Curran Associates, Inc., Red Hook, NY, USA, 9459–9474.

[30] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. 2010. A Contextual-Bandit Approach to Personalized News Article Recommendation. In *Proceedings of the 19th International Conference on World Wide Web*. Association for Computing Machinery, New York, NY, USA, 661–670. https://doi.org/10.1145/1772690.1772758

[31] Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. TruthfulQA: Measuring How Models Mimic Human Falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Dublin, Ireland, 3214–3252. https://doi.org/10.18653/v1/2022.acl-long.229

[32] John D. C. Little. 1961. A Proof for the Queuing Formula: $L = \lambda W$. *Operations Research* 9, 3 (1961), 383–387. https://doi.org/10.1287/opre.9.3.383

[33] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173.

[34] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *The Twelfth International Conference on Learning Representations*. OpenReview.net, Vienna, Austria. https://openreview.net/forum?id=zAdUB0aCTQ

[35] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. 2025. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, Albuquerque, New Mexico, 1160–1183. https://doi.org/10.18653/v1/2025.findings-naacl.65

[36] Garm Lucassen, Fabiano Dalpiaz, Jan Martijn E. M. van der Werf, and Sjaak Brinkkemper. 2016. Improving Agile Requirements: The Quality User Story Framework and Tool. *Requirements Engineering* 21, 3 (2016), 383–403.

[37] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., Red Hook, NY, USA, 46534–46594. https://doi.org/10.52202/075280-2019

[38] Daniel Méndez Fernández and Stefan Wagner. 2015. Naming the Pain in Requirements Engineering: Contemporary Problems, Causes, and Effects in Practice. *Information and Software Technology* 57 (2015), 575–590.

[39] Luc Moreau and Paolo Missier. 2013. *PROV-DM: The PROV Data Model.* W3C Recommendation. World Wide Web Consortium. https://www.w3.org/TR/prov-dm/

[40] Bashar Nuseibeh and Steve Easterbrook. 2000. Requirements Engineering: A Roadmap. In *Proceedings of the Conference on the Future of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 35–46.

[41] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. 2022. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. Curran Associates, Inc., Red Hook, NY, USA, 27730–27744. https://doi.org/10.52202/068431-2011

[42] Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. 2000. A Model for Types and Levels of Human Interaction with Automation. *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans* 30, 3 (2000), 286–297.

[43] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *The Twelfth International Conference on Learning Representations*. OpenReview.net, Vienna, Austria. https://openreview.net/forum?id=GEcwtMk1uA

[44] Per Runeson and Martin Höst. 2009. Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Software Engineering* 14, 2 (2009), 131–164.

[45] Daniel J. Russo, Benjamin Van Roy, Abbas Kazerouni, Ian Osband, and Zheng Wen. 2018. A Tutorial on Thompson Sampling. *Foundations and Trends in Machine Learning* 11, 1 (2018), 1–96. https://doi.org/10.1561/2200000070

[46] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2022. ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Seattle, United States, 3715–3734. https://doi.org/10.18653/v1/2022.naacl-main.272

[47] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36. Curran Associates, Inc., Red Hook, NY, USA, 68539–68551. https://doi.org/10.52202/075280-2997

[48] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36. Curran Associates, Inc., Red Hook, NY, USA, 8634–8652. https://doi.org/10.52202/075280-0377

[49] Klaas-Jan Stol and Brian Fitzgerald. 2018. The ABC of Software Engineering Research. *ACM Transactions on Software Engineering and Methodology* 27, 3, Article 11 (2018), 51 pages.

[50] Margaret-Anne Storey, Neil A. Ernst, Courtney Williams, and Eirini Kalliamvakou. 2020. The Who, What, How of Software Engineering Research: A Socio-Technical Framework. *Empirical Software Engineering* 25, 5 (2020), 4097–4129.

[51] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction* (2 ed.). The MIT Press, Cambridge, MA, USA. https://mitpress.mit.edu/9780262039246/reinforcement-learning/

[52] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 1–7.

[53] Kees M. van Hee and Wil M. P. van der Aalst. 2004. *Workflow Management: Models, Methods, and Systems.* The MIT Press, Cambridge, MA, USA. https://mitpress.mit.edu/9780262720465/workflow-management/

[54] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. Curran Associates, Inc., Red Hook, NY, USA, 24824–24837. https://doi.org/10.52202/068431-1800

[55] Frank Wilcoxon. 1945. Individual Comparisons by Ranking Methods. *Biometrics Bulletin* 1, 6 (1945), 80–83. https://doi.org/10.2307/3001968

[56] Claes Wohlin. 2014. Guidelines for Snowballing in Systematic Literature Studies and a Replication in Software Engineering. In *Proceedings of the International Conference on Evaluation and Assessment in Software Engineering*. Association for Computing Machinery, New York, NY, USA, 1–10.

[57] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering.* Springer, Berlin, Germany.

[58] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *Conference on Language Modeling (COLM)*. OpenReview.net, Philadelphia, PA, USA. https://openreview.net/forum?id=BAakY1hNKS

[59] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36. Curran Associates, Inc., Red Hook, NY, USA, 11809–11822. https://doi.org/10.52202/075280-0517

[60] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Kigali, Rwanda.

A SCORING MODEL SANITY CHECKS UNDER DETERMINISTIC PERTURBATION

Table 10 summarizes the formula-generated profiles for the internal and public-input checks. The internal check applies seeded perturbations to reused runtime-derived inputs; the public-input check applies the same formulas to 300 SWE-bench Lite issue texts [23]. B0–B3 identify scoring profiles in this table. Their composite is the equally weighted mean of the quality, efficiency, and governance indices on a 0–100 scale. These rows check the scoring implementation and are separate from the executed baseline comparison in Section 5.3.

Table 10. Scoring model sanity checks under deterministic perturbation. All entries are formula-derived means.

Input	Profile	Rows	Quality	Efficiency	Governance	Composite
Seeded internal units	B0	5,000	60.17	30.28	35.18	44.96
	B1	5,000	59.66	47.01	36.45	50.06
	B2	5,000	73.76	52.33	41.97	59.38
	B3	5,000	88.18	66.97	68.76	76.96
Public issue text	B0	300	60.39	30.62	35.21	45.16
	B1	300	57.72	48.20	35.34	49.27
	B2	300	79.36	48.39	42.79	60.93
	B3	300	89.92	66.65	69.72	77.89